DNSSEC: A Study Into The Signing and Validation of Signatures

by

Steven Dormady

Advised by

Dr. Mohamed Aboutabl

An Independent Study Project

Department of Computer Science

James Madison University

Department of Computer Science

May 2026

Contents

Abstract

This study dives into the Domain Name System and the vulnerabilities that come with plain DNS. These vulnerabilities, like DNS Spoofing and Hijacking, usher in questions about how they can be prevented and mitigated. These attacks are discussed and demonstrated in this paper. This is where the Domain Name System Security Extension, or DNSSEC, comes into play. DNSSEC prevents these attacks by providing authentication and integrity to the data. The DNS data is hashed, providing integrity in the way that if any data was changed, the hash would be different, and signed, brining authentication knowing that only the DNS server could've signed the record with the private key. OpenSSL is used to do this, and I used SHA256 for hashing and RSA for the public key infrastructure. I implemented the signing process in C using these functions and manually built the data to be signed, generating my own signatures, verifying the existing signature, and comparing the binaries of the two to make sure they were valid.

Introduction

1.1 Motivation

DNS by itself is insecure and vulnerable to attack. With DNSSEC, these vulnerabilities are contained and mitigated.

My personal motivation for this study was a hand on security project, advised by Dr. Aboutabl, who has become one of my personal favorite professors here at JMU. His Information Security class helped further my interest for security, and when he mentioned this study, I knew I had to seize the opportunity.

1.2 Problem Statement

The problem we are trying to solve here is a dive into DNS Security Extension, or DNSSEC, and how it changes DNS. Using a testbed network of virtual machines on a virtual network, I was able to use Bind9 to be able to set up the DNS environment, then convert it to use DNSSEC. In this study of DNSSEC, I will be looking at the algorithms and data structures used in DNSSEC, and how they are used to secure DNS. I will also be looking at the performance of DNSSEC, and how it compares to DNS.

1.3 Contributions

I contributed the research and implementation of the DNS testbed environment, the algorithms and data structures used in DNSSEC, and the performance analysis of DNSSEC. I also contributed to the writing of this thesis, and the presentation of the results. I also manually wrote the code to generate signatures and validate them, which is a good way to test the validation process, because I can control the data that is being signed and verified. Not only that, but I am also creating the signature myself to prove that the received signature is valid as well. I used Draw.io to create my diagrams.

Dr. Mohamed Aboutabl, my advisor, contributed in providing inspiration when I was stuck, and providing guidance on how to approach the project. He also provided feedback on the writing of this thesis, and the presentation of the results.

1.4 Virtual Machine Fundamentals

I have never dealt directly with Virtual Machines before, and building a network of them was something I had never really thought about. However, the thought excited me and it seemed like a great opportunity to learn a new skill. Setting up the first one was a bit of a learning curve, but after setting up the first one to the specifications I wanted, I was able to clone the rest as needed.

The software I used to set up the testbed network for my DNSSEC program was VMWare Workstation. This software made it very easy to set up the virtual machines and network. Having a built in option to create a virtual network was very nice. I was also able to connect to the internet and download the necessary tools I needed, like Bind9, VMWare Tools, Wireshark, gcc and more. I also chose to use git for version control and being able to edit the code on multiple devices if needed.

1.5 Testbed Virtual Network

On VM Workstation, it is very easy and simple to set up a Virtual Network. Under the ”Edit” tab in the menu, I was able to create a new vmnet and customize it to my needs. I made a custom network, isolated from the outside networks. The IP I used was 192.168.107.X, with a subnet mask of 255.255.255.0. The 192.168 is an IP reserved for local networks, so this was perfect for me to to use. There was an option to enable or disable DHCP service. Enabling it meant that you could let DCHP assign addresses to machines on the network. For me, I wanted to statically and manually assign IPs without and DHCP intervention, so I deselected this option to have more control in my environment.

1.6 Bind9

I installed Bind9 on all of the VMs in the Virtual Network. The version I installed was 9.18.39. I decided on Bind9 because it was a good option to be used on Ubuntu machines and gave me the control I wanted.

Introduction to the Domain Naming System

2.1 DNS Background

DNS started as ARPANET, which was very inefficient and used a centralized hosts file to map the domain names to IP addresses. When a change was made, it had to be redistributed to all of the systems. With number of users and systems needed increasing, this became very hard to maintain, which led to DNS today.[1]

In my research about DNS, I learned a lot about what happens behind the scenes with DNS Servers. Cloudflare called it, ”DNS is the phone book of the internet.”, and like a phonebook, you look up a name (domain name) and get someone’s phone number (IP address).[2] All network systems operate with network addresses, such as IPv4 and IPv6. More or less, DNS translates domain names, like www.example.com, to IP addresses, like 134.126.126.99 (jmu.edu), so browsers can load internet resources.

The DNS naming structure is organized hierarchically in a tree-like format. Starting at the root of the tree, there is the Root Domain, which is where real world DNS begins. It delegates authority to the Top-Level Domains, or TLDs, which delegate further to Second-Level Domains, or SLDs, and continue onward, spreading out like a tree. Below in figure 2.1 is a diagram demonstrating this hierarchy.[1]

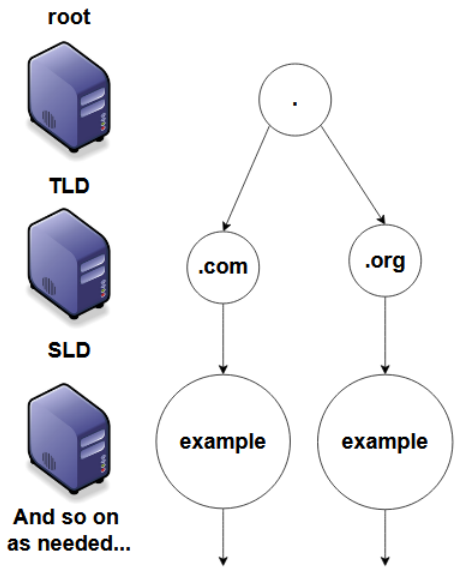


Figure 2.1: Root Delegation Hierarchy

A domain is the label of a node in the tree. A dot in a domain name means that it must traverse to another layer to get more information. Like in the above example, we start with ”example”, being at the Second-Level domain, then traverses to ”.com” at the Top-Level domain, then finally the root, ”.”, server. Put it all together and you get example.com., with the end period making a Fully Qualified Domain Name, or FQDN. This period is usually hidden behind the scenes, and as users we do not use it. Root servers is the beginning of all DNS name resolutions, which makes the name root make sense. Like a tree, the DNS tree needs its roots to work. However, it helps to see the root domain being reached in this instance. There are 13 root servers due to historical reasons of how many addresses could be put into a 512 byte UDP message.[1]

2.1.1 DNS Resource Records

These DNS records are the most common records used in building a DNS server. These records are all in db files, which are database files used for each zone defined. For example, the zone ”.lab” will have a db.lab file. You can name it whatever, as long as it aligns with the named.conf.local zone path defined for the zone. It is best practice to have the domain

name as the db files extension. For both the A and NS records, their line begins with an @ symbol. Below is an example of my db.lab file for the lab zone on VM1.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.lab.

ns1     IN      A       192.168.107.129

jmu.lab.        IN      NS      ns1.jmu.lab.
ns1.jmu.lab.    IN      A       192.168.107.128
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

Figure 2.2: The db.lab file for the lab zone.

Start Of Authority Records

SOA records, or ”Start Of Authority” records, show the domain name that the specific db file has authority over. There are several sub records within. The MNAME is the domain name of the name server of the original or primary source of data for the zone. Like a .com, or for this study, .jmu and .lab. These are typically preceded with an ns1, which will come into play later with Nameserver Records later. The next record is a RNAME, which is the Responsible Person record. For this project, I have root.jmu or root.lab for the root having responsibility. The next record is the serial number, which Bind uses to load different serial versions of the zone. This record has the space to be an unsigned 32 bit number. It should be updated every time there is a change to the db file. Some developers like to use the date it was edited on to help keep track of when it was last edited. Next, the refresh record is a 32 bit time interval before the zone should be refreshed. The next record is the retry record,

which is another 32 bit time interval that would run out before a failed refresh should give up. Lastly, the expire record defines another 32 bit time value that gives an upper bound on the time interval that can run before the zone is no longer authoritative. Below is an example of an SOA record for lab zone in figure 2.3.[3]

```
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
```

Figure 2.3: An example of the SOA record in the parent servers db.lab, representing the start of authority for the zone.

Address Records

A records are address records, "A" for short. These are the records you get when a domain maps to an IP address of a given domain. This is the most fundamental type of DNS record. There are also "AAAA" records, which are for IPv6 traffic, but for this study we will only focus on IPv4 IPs. These records enable a user's device to connect with and load a website, or whatever is at the IP address.[4] As seen below, in figure 2.4, when querying for an ip at one of my DNS Servers, I am returned with the IP address of that record.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab A +short
192.168.107.2
```

Figure 2.4: An example of digging for an A record, the short flag used to only give me the ip.

Below, figure 2.5, is an example of what these records look like.

```
bc          IN      A       192.168.107.1
computer            IN      A       192.168.107.2
```

Figure 2.5: An example of A records in the parent servers db.lab, computer.lab and bc.lab

Name Server Records

NS records stand for Name Server records, and indicate which DNS server is authoritative for that domain. Like in the SOA, ns1.lab shows which server is authoritative for the .lab domain. Domains can have multiple NS records, like .lab has, which indicate primary and secondary nameservers for domains. For instance, in db.lab, there are multiple NS records, one pointing the the server itself and another pointing to the secondary server for the .jmu.lab domain.[5] See figure 2.6 for an example.

```
@           IN      NS      ns1.lab.

ns1         IN      A       192.168.107.129

jmu.lab.            IN      NS      ns1.jmu.lab.
ns1.jmu.lab.        IN      A       192.168.107.128
```

Figure 2.6: An example of NS records in the parent servers db.lab

2.2 The Testbed DNS Environment

There are numerous similarities between DNS with the internet and this testbed study. The bridge between the user and DNS servers is the DNS resolver or name resolution. In the project, the client digs at a server, and gets a response. This is a higher level abstraction, as there are no websites mapped to the IPs on my DNS servers, just more local IPs which could be a website. There is also no official ”.” or root domain, but my ”root” domain is lab., which is also my top level domain. This is seen in figure 2.7. In this setup, authority is held in the parent vm and passed downward onto the child server. In real DNS, a recursive resolver is used and traverses the different levels of domains.

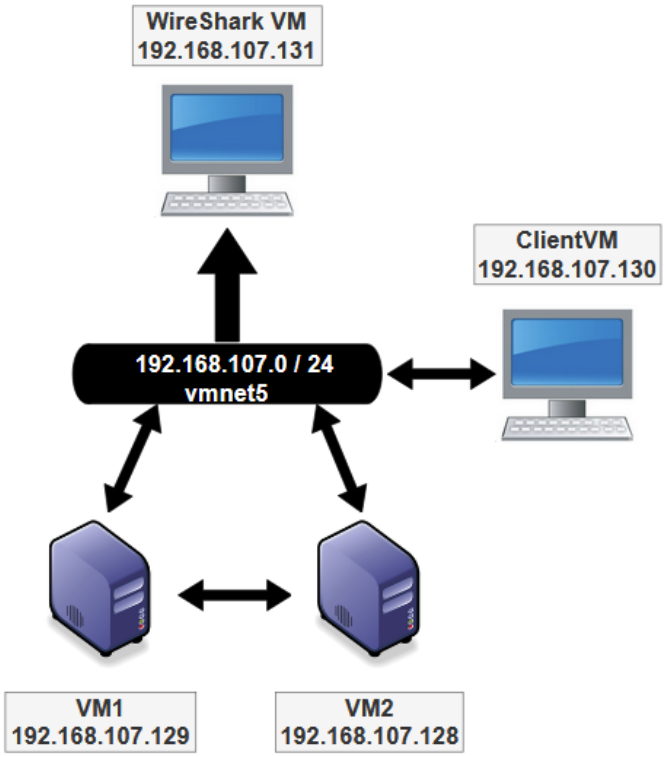


Figure 2.7: This is a diagram of the setup of the virtual environment, with inspiration of the layout coming from Dr.Aboutabl

2.2.1 Testbed Environment Zones

In the environment, there are different zones between the two name servers. As seen in figure 2.8, the parent zone holds .lab, with an extension to the child zone, .jmu. To be able to query the child, you need a domain name, followed by .jmu.lab. A full example of a valid query would be like this: quad.jmu.lab. Quad is the final piece, and this domain maps to 192.168.107.64. You can query the child server directly and receive responses, or you can query it through the parent server.

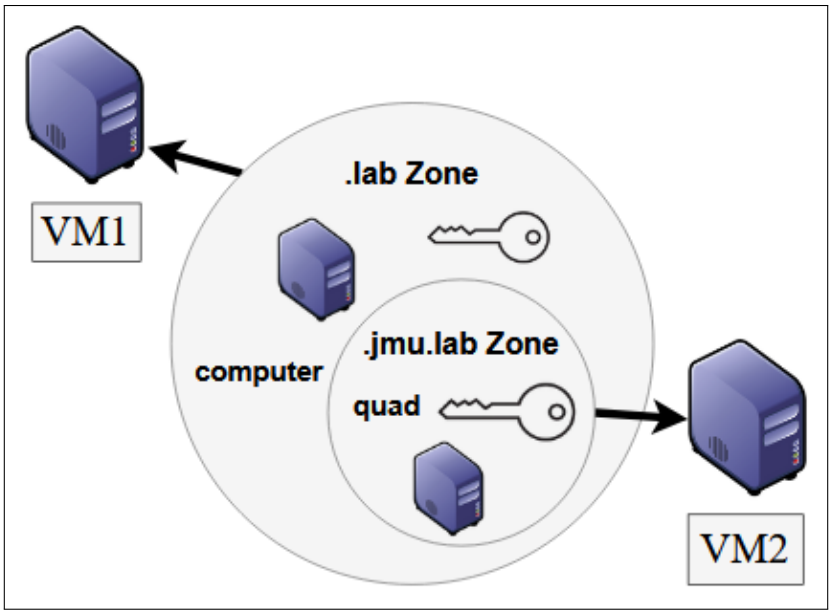


Figure 2.8: This is a diagram showing the zones and their relationship.

In this study, both the parent and the child server have authority over their own domains, and you can also traverse the tree to query the parent for information on the child server, as seen below in figure 2.9. There are also no IPV6 records, as this is disabled in the configuration of the servers.

```
;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.jmu.lab.

;; ADDITIONAL SECTION:
ns.jmu.lab.             86400   IN      A       192.168.107.128

;; Query time: 1 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Tue Feb 24 22:57:16 EST 2026
;; MSG SIZE  rcvd: 102

steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.128 quad.jmu.lab A +short
192.168.107.64
```

Figure 2.9: Querying the parent for information about the child, the querying the child with that information.

2.3 DNS Vulnerabilities

DNS without DNSSEC is very insecure and vulnerable to a plethora of attacks. Without DNSSEC, queries are readable to any person on the internet. An attacker could easily intercept this traffic and replace it with malicious content. This is where DNSSEC comes into play, helping to mitigate and prevent many attacks from occurring by providing integrity and authentication to the data. It also proves that the information is coming from a trusted, authoritative source.

2.3.1 DNS Spoofing/Cache Poisoning

This is an especially vulnerable part of DNS without DNSSEC. This attack is a textbook man-in-the-middle attack, when the attacker intercepts a query and replaces it with their malicious information. DNS spoofing is when an attacker inserts their malicious address in place of a cached address. This attack is using DNS Hijacking in this instance, to redirect to what the attacker wants them to see. They pretend to be a DNS nameserver and forge a reply, and with UDP it makes it even easier for this attack to occur. Cache poisoning is DNS Spoofing without having to hijack the query. The systems involved with DNS, like servers, computers, and routers can cache DNS lookups for quicker recall. An attacker can poison the cache by replacing an entry with a spoofed domain. This spoofed domain gets accessed until the cache gets refreshed and the spoofed site gets replaced.[6]

There are a few criteria that need to be met in order for this attack to be successful. One is that the domain cannot be saved in the cache already, meaning that the attacker cannot directly modify the cache in this way and would have to wait for the Time To Live to expire, or the record is not already in the cache. The attacker also has to guess the Query ID, which used to be easier to do, back when the Query ID was just incremented by one between queries. Now, the Query ID is random, which makes it much harder to guess. The attacker also has to guess the source port of the query, which is also random, making it even harder

to guess. Query IDs also only used to be 16 bits long, which meant that brute forcing 65,536 possible combinations of ids wasn't that difficult. Once this information is known by the attacker, they can forge a response much easier and avoid detection [7].

To actually perform this attack, the attacker will flood the nameserver searching for the answer with an iterative attack of forged responses with various query IDs and source ports. If the attacker is able to guess the correct query ID and source port, then the forged response will be accepted by the nameserver and cached for future use, seen in figure 2.10 below.

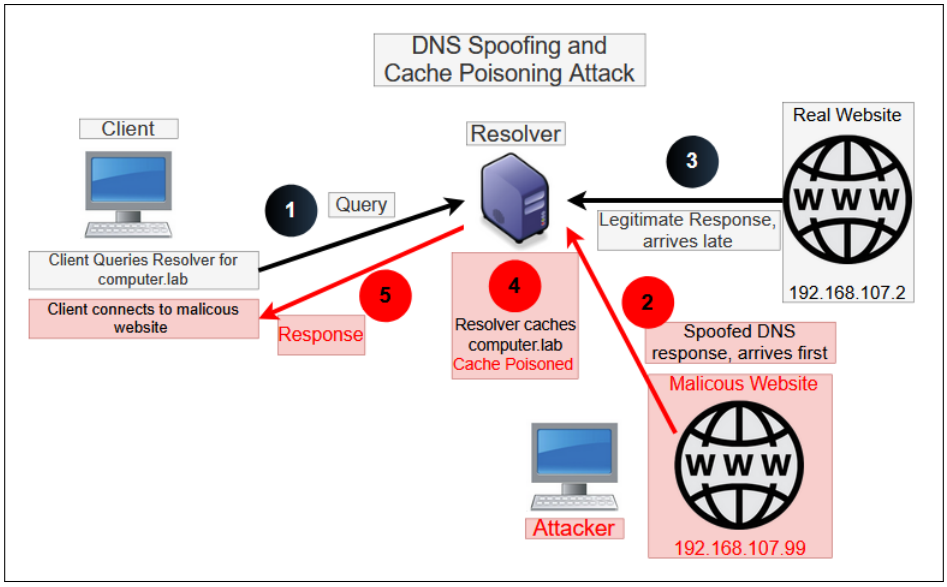


Figure 2.10: An example of DNS Spoofing, where the attacker intercepts a query and replaces it with their malicious information.

1. The client makes a query to the DNS server, asking for the IP address of a domain name.

2. The attacker intercepts this query and sends a forged response back to the client, containing the IP address of a malicious website instead of the legitimate one.

3. The DNS server receives the forged response and the legitimate information arrives late.

4. The client receives the forged response and updates its cache with the malicious IP

address. The client is now unknowingly directed to the attacker's website whenever it tries to access the legitimate domain name.

5. Finally, the attacker can use this access to the client's machine to steal sensitive information, such as login credentials or financial data, or to distribute malware.

With modern DNS implementations, there are various mitigations in place to prevent this attack from being successful. For example, randomizing the Query ID and source port makes it much harder for attackers to guess the necessary information to forge a response. Additionally, having a wider range of possible ports and longer Query IDs makes it even more difficult for attackers to successfully perform this attack. However, without DNSSEC, there is still a risk of this attack being successful, especially if the attacker has access to the network traffic and can intercept queries and responses.

2.3.2 DNS Hijacking

DNS Hijacking occurs when an attacker tries to redirect where the client is going with their query, by gaining root access to the server. They can perform a man-in-the-middle attack and take the query and replace it with their IP, causing the user's machine to directly communicate with the attacker. They could also give an IP that delivers a malicious payload, or redirects the client to a malicious website, allowing the attacker to steal sensitive information. These are some of the most basic types of attacks when it comes the DNS hijacking[8]. In figure 2.11 below, it shows a diagram of how this attack works.

1. The attacker hijacks the DNS server, either by exploiting vulnerabilities in the server software or by gaining unauthorized access to the server. This allows the attacker to control the responses that the server sends back to clients, in this case changing the ip of computer.lab to the attacker's IP.

2. The client makes a query to the DNS server, asking for the IP address of a domain name.

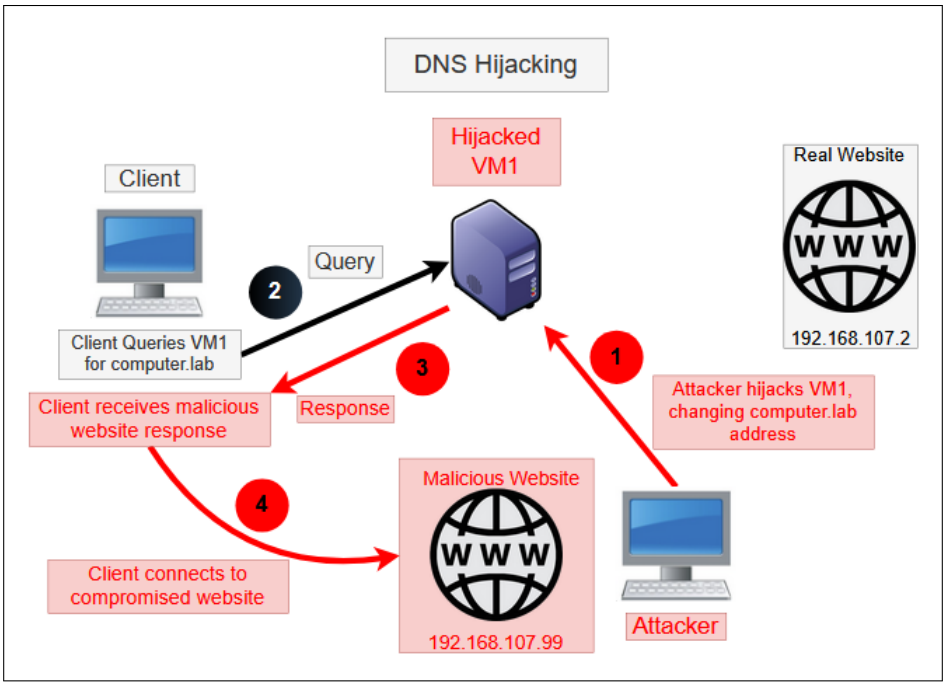


Figure 2.11: An example of DNS Hijacking, where the attacker redirects a query to a malicious server.

3. The DNS server sends the malicious response back to the client, containing the IP address of a malicious website instead of the legitimate one.

4. Finally, the client unknowingly connects to the attacker's website, which can be used to steal sensitive information, such as login credentials or financial data, or to distribute malware.

2.4 Simulated DNS Attacks In The Testbed Environment

To show these attacks, I simulated them in the testbed environment. I assumed the role of the attacker and was able to hijack VM1, the root dns server of the virtual network. I was able to gain root access to the server, allowing me to modify it however I pleased. As seen in the figure below, the first step was to take control of the parent server and then change the db.lab file to have a malicious zone, which is shown in figure 2.12. Instead of being redirected to jmu.lab. NS, users are now redirected to the attacker's ns.evil.lab. This is a

form of DNS Hijacking, and it also allows for DNS Spoofing and Cache Poisoning to occur. After hijacking the server, I was able to modify any record. Instead of just modifying the NS record, I could also modify an A record to have a malicious IP address, which would redirect users and resolvers to a different malicious IP address, making the attack complete and compromising the user.

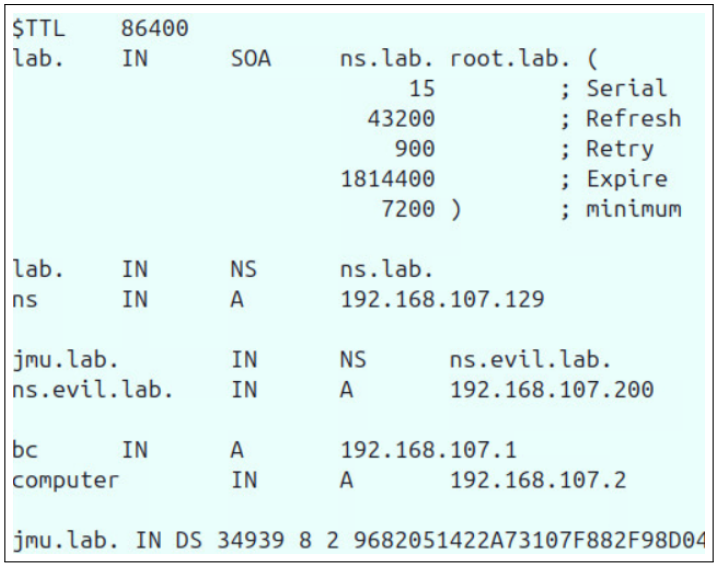


Figure 2.12: After hijacking the parent server, I was able to change the db.lab file to have a malicious zone, which is shown above.

After injecting this into the DNS server, I queried the parent server for the NS record of jmu.lab, and I was returned with the attacker's NS record, which is ns.evil.lab. This is shown in figure 2.13. Without DNSSEC, there is no way for the client to know that this is not the real NS record, and they will be redirected to the attacker's server.

In this query, the client is asking for the NS record of jmu.lab, and instead of getting the legitimate NS record, they are getting the attacker's NS record. This is a successful DNS Hijacking attack, and it also allows for DNS Spoofing and Cache Poisoning to occur. These attacks are successful without DNSSEC, but later in Simulated Attacks with DNSSEC Enabled

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 jmu.lab. NS +norec

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 jmu.lab. NS +no
rec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52499
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 43a03a13c7ad7e9a0100000069b49377d30f4e3f16062eae (good)
;; QUESTION SECTION:
;jmu.lab.                       IN      NS

;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.evil.lab.
```

Figure 2.13: Successful injection of malicious DNS data.

Introduction to DNS Security Extension

3.1 DNSSEC Background

DNSSEC, or Domain Name System Security Extension, adds a layer of security to the otherwise vulnerable DNS. DNSSEC adds security by using cryptographic signatures with public key cryptography, with the private key signing it and the public key verifying. DNSSEC also implements hashing, adding integrity to the data and the signing provides the authentication, knowing that only the private key from the server could've signed that data.

There is an additional flag with dnssec queries, the "ad" flag. This flag is passed along with successful DNSSEC queries, meaning "Authenticated Data". When you query for a domain with dnssec enabled, receiving this flag back instead of a SERVFAIL status means that the data was successfully authenticated. This is normally added by the resolver before being passed onto the client, meaning the resolver says it is authenticated and can be trusted on that last part of the data's journey to the client machine.

You will not see it in the below screen shots, and that is because there isn't any validation going on yet. Seeing the records is enough for this part of my project, whereas the next chapters cover the code I wrote to validate these records. Later, when I use the `delv` command, the data is validate by a resolver.

Below, figure 3.1, is db.lab, but now set for DNSSEC. The directory was changed to give the

zone owner write permissions without requiring root access. Originally, DNS configuration and the db files were in /etc/bind/, but now they moved to /var/lib/bind/ to allow for more freedom and not having to deal with bind ownership. Changing the zone configuration to point to the new db file allows the server to write into this directory. As you can see below, there are a number of changes to the db file. Gone are the @ symbols preceding lines, and there is a new minimum time record. This sets the minimum Time-to-Live (TTL) for records, especially for the Next SECure (NSEC) records. There is still a SOA, NS and A records, which have minimally changed from regular DNS. At the bottom of the figure, you can see a Delegation Signer record, which came from the child zone. This is an important part of the Chain of trust, allowing the parent to verify the child is the child.[9]

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F1
42C2
```

Figure 3.1: The new DNSSEC db.lab

3.2 New DNS Resource Records

With the regular DNS records previously discussed, there are some new records that must be taken into account. Regarding the Zone Signing Key and the Key Signing Key, the algorithms I used for the keys was RSASHA256, this is a good algorithm for signing and one I am familiar with. The ZSK signs the records from the zone, and the KSK signs the ZSK. The biggest takeaways from the new resource records is that every record gets

signed, and some records exist to just provide more security and prevent attacks, like the Next SECure Records

3.2.1 Resource Record Set

This record is not an explicit record, rather a descriptive term to describe records that are grouped together. For instance, if I query for my parent server for DNSKEYs, I receive the RRSet that is signed, seen below in figure 3.2. These keys are signed together as a set by the Key Signing Key, whose name is self-explanatory for what its purpose is. [10]

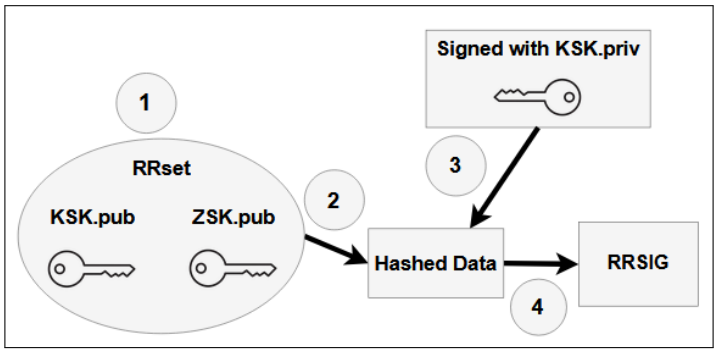


Figure 3.2: This query shows a dig at the parent for the DNSKEYs

Canonical Ordering

In some queries, there are multiple domain names that map to different IPs. Canonical ordering exists because DNSSEC needs to have the same deterministic order for similar records across the board, or else these Resource Record Sets would equate to different hashes. Canonical is in order of most significant labels, like lab has one label, also the root zone in my environment, and is more significant than jmu.lab. All uppercase letters are also converted to lowercase and are in alphabetical order. If two domains share the exact same uncompressed name, they are grouped in an RRSet, in order of their RDATA, which is the value that the domain holds, like flags, key tags, IP addresses, and so on.[10]

Below is an example of an address query that has two domains with two different IP's, mapped together here in the same Resource Record Set.

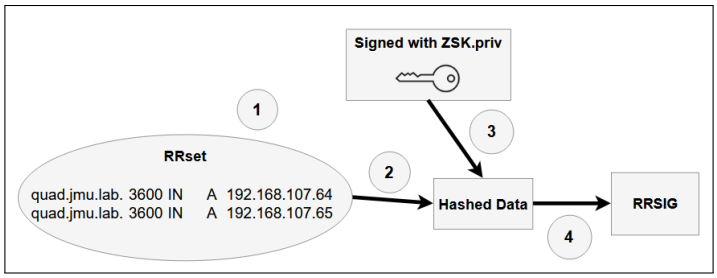


Figure 3.3: This diagram of an address query shows canonical ordering in action, grouping two domains in the same set in a specific order.

3.2.2 Signature Resource Record

The Resource Record Signature, or RRSIG record, is used to verify the answers that are received. These are attached to every answer that is sent with DNSSEC. The signatures are used by recursive name servers to verify that the answer to the question is valid. [11] Below is an example of an address query with a signature, figure 3.4.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab +dnssec +multiline

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 computer.lab +dnssec +multiline
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19718
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 3df3cdc0dcb579af0100000069a5d721e88ca1e9eeb51c37 (good)
;; QUESTION SECTION:
;computer.lab.          IN A

;; ANSWER SECTION:
computer.lab.           86400 IN A 192.168.107.2
computer.lab.           86400 IN RRSIG A 8 2 86400 (
                                20260310233437 20260225023500 34104 lab.
                                GkosYckWQnxLuVJhdzymzDDRZuS+hBs5zzsdmrBCFkc3
                                4hdxuO8bbGjPQfkJTB9fXyov9xc6M1rDQko8+qzgD+9d
                                Kuextffp+oU3kiOLiJosuEUlDciTkVR/u4nm0T84STDN
                                RmK1yjPtp+hb4ZK0eLTIm3Ksqx/xQsT6Pl6Gl/Q= )

;; Query time: 0 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Mon Mar 02 13:29:53 EST 2026
;; MSG SIZE  rcvd: 248
```

Figure 3.4: Address record with a RRSIG from the ZSK

There is a lot of information in the fields that come with an RRSIG resource record. After the computer.lab domain name and the TTL, 86400, the first field is the type, being a RRSIG for an address record. After that, there is the algorithm field, 8 standing for RSASHA256. The next field is the labels field, which represents the count of labels in the domain name,

excluding the root. The next field is the original TTL of the RRset, which is the value it is given in the authoritative zone. The field that follows the TTL is the signature expiration and inception fields. These are 32-bit unsigned integers, represented in the number of seconds since January 1st, 1970, or like the format above, being YYYYMMDDhhmmss, years, months, days, hours in 24 hour format, minutes and seconds. The latter format is distinguishable by having exactly 14 digits. The seconds option with be a 32-bit integer that won't be longer than 10 digits. The inception date specifies when it was created, and the expiration date being when it is no longer valid. Next is the key tag field, which holds the key tag of the signing key, in this instance the ZSK for the lab zone.

This is shown below in figure 3.5.

Field	Size	Description
Type Covered	2 bytes	RR type being signed (A=0x0001)
Algorithm	1 byte	8 = RSASHA256
Labels	1 byte	Number of labels in owner name
Original TTL	4 bytes	TTL from the RRSIG record
Sig Expiration	4 bytes	Unix timestamp, big endian
Sig Inception	4 bytes	Unix timestamp, big endian
Key Tag	2 bytes	Key tag of signing ZSK
Signer's Name	variable	Zone name in DNS wire format
Owner Name	variable	RR owner name in DNS wire format
Type	2 bytes	Same as type covered
Class	2 bytes	IN = 0x0001
TTL	4 bytes	Must match original TTL exactly
RDLENGTH	2 bytes	Length of RDATA
RDATA	variable	The actual record data (e.g. IP address)

Figure 3.5: A diagram showing the different fields of the RRSIG record, and what they represent.

All of the metadata inside the RRSIG Record, from the type to the keytag, is first in th buffer to get signed, then the resource record sets information gets appended to the meta data and then hashed, then signed with the correct key for the type of record. This creates the signature which is the last piece of the RRSIG resource record puzzle.

3.2.3 DNSKEYs In General

DNSKEYs are the record that are used to sign and validate RRsets. These keys use public key cryptography, storing the public keys in DNSKEY RRs. These keys are used in different ways, and there are two different keys that we care about in this study, which are the Zone Signing Key and the Key Signing Key. The identifying type number that this RR is a DNSKEY is 48.[10]

The general format for these RRs is as follows, figure 3.6

Figure 3.6: This query shows a dig at the parent for the DNSKEYs

These keys are stored in /var/cache/bind/, and are generated with the `dnssec-keygen` command. There are two parts of these keys, a ".key" and ".private", with the zone name and key tag in the name. There is an example of these stored records below. The keys are generated with the algorithm specified in the dnssec policy, which is RSASHA256 in this case. Generally, the KSK is generated with a larger key size than the ZSK, because the KSK is used to sign the ZSK, which is used to sign the records. The KSK is more important to protect, because if it is compromised, then the ZSK can be compromised, and then all of the records can be compromised. The ZSK is used more frequently, so it is more likely to be compromised, but it is also easier to replace than the KSK.

The Zone Signing Key

The Zone Signing Key, or ”ZSK” for short, is the key that is used to sign data from a zone. It signs all records except for RRset that are related to keys. The ZSK has the 256 flag, indicating that it is the ZSK. This key has a shorter Time To Live, or TTL, because it is sent out and used much more frequently than the Key Signing Key. Seen later under DNSSEC Policy, I set the time to live for the ZSK to be 60 days, where as the KSK is set to be a year. From what I saw, this is pretty standard.

The Key Signing Key

The Key Signing Key, or ”KSK”, is used to sign what the ZSK does not sign: key-related RRsets. This key serves another purpose, as it is the bridge between a parent and a child zone. The flag for the KSK is 257, indicating that this key is the KSK. Like the Zone Key, this key also follows the public key encryption scheme. This key pair is meant to protect the ZSK for the zone being queried.

3.2.4 The Delegation Signer

The Delegation Signer Record, or DS record, is used to ”vouch” for another zone. The parent holds a copy of this in their zone, and when asked to query about the child zone, it can verify that the response comes from the child. This record is a hash that is stored of the child zone’s KSK, and the parent can use this hash to verify that the child zone is the child zone. This is an important part of the Chain of Trust, allowing the parent to verify the child is the child, ensuring that the responses they receive can be trusted.[9]

These records are generated with the `dnssec-dsfromkey` command, which takes in the child zone’s KSK public key and generates the DS record that can be stored in the parent zone. The output from this command gets pasted directly into the parents zone file, and then the parent can use this to verify the child zone’s KSK. Below is an example of a DS

record and the command used to generate it, figure 3.7. After adding this delegation signer record to the parent zone, I had to run `rndc reload` to reload the zone.

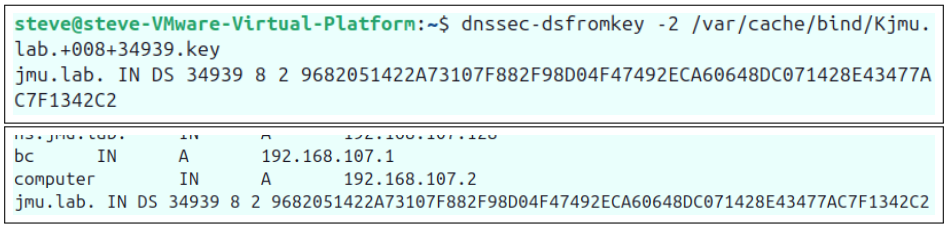


Figure 3.7: The first image is the command used to generate the DS on the child server, then the output in the parent zone file.

3.2.5 The Next Secure Resource Record

The Next Secure Record, or NSEC record, is used to prove that a name does not exist. This is used in the case of a query for a non-existent domain, where the server can respond with an NSEC record that proves the non-existence of the domain. This is done by including the next existing domain in the zone, and then signing this record with the ZSK. This way, the client can verify that the non-existent domain does not exist, and that the response is authentic.[12] An example of an NSEC record is shown below in figure 3.8.

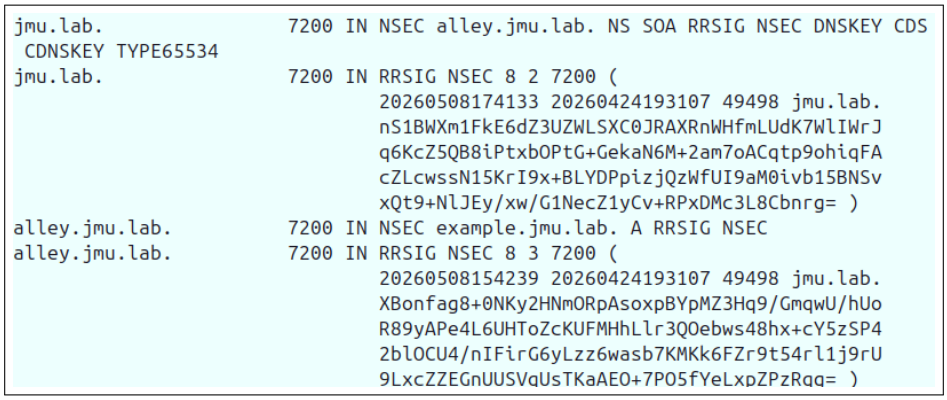


Figure 3.8: In this query, I tried to ask about a zone that didn't exist and got this response.

NSEC3 records were also introduced after NSEC records, which are used to provide more security against zone enumeration attacks. These records use a hash of the domain name, rather than the domain name itself, to provide more security against attackers trying to

enumerate the zone. This way, even if an attacker tries to enumerate the zone, they will only see the hashed names, which are not easily reversible.[12]

3.3 The Chain Of Trust

The Chain of Trust is how different levels of servers know they can trust each other. There must be trust through the whole process of the query, otherwise there is a vulnerable spot for intrusion by an attacker. The trust starts with a fitting name, the trust anchor.[13] An example of a trust anchor is below in figure 3.9. This record is stored on the client, and is a copy of the parent zone's KSK. This continues the Root Delegation Hierarchy, described earlier in figure 2.1, just in terms of these lab zones and DNSSEC enabled.

```
trust-anchors {
        lab. static-key 257 3 8 "AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqMF19e
5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4BRP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF
7oWmcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZLkalddzkc0KuvwAtmGsOTT7lkn3sx59JgWG
SYiL+3FNZFSZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr27TtP3LGJiBhcyMeJRT3i4bOBbCt
wGelA6xq5i4k1fg2+R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=";
};
```

Figure 3.9: An Example of my Trust Anchor

The trust anchor is the root of the trust, and the client can use this to verify the parent zone's KSK, which then can be used to verify the ZSK, which then can be used to verify the records from the zone. This chain continues onto the child zone, where the parent can verify the child zone's KSK with the DS record, and then use that KSK to verify the ZSK, which then can be used to verify the records from the child zone. This is a repetitive process, where the parent can verify the child, and then the child can verify the next child, and so on. [14] A figure of the chain of trust is shown below in DNSSEC In the Environment, figure 3.15. Maintaining the chain of trust also requires key management, as the keys do not last forever and need to be rotated out, as seen in Key Rollovers.

3.4 Key Rollovers

I ran into an issue when trying to run my validation code, where my ZSK had outlived its 60 day TTL. I was testing my validation code, seen in Programming the Signature and Validation of Several DNSSEC Resource Records, and noticed it was failing where it used to succeed. I decided to check out my keys and sure enough, they had expired and could no longer be used. I then pulled the newly generated keys from the servers and tried to query for new signature records. However, I noticed there were two signatures attached to zone records and three keys being signed in a DNSKEY query, seen in figure 3.10 and figure 3.11 below. This is because the old ZSK is still active but wasn't used for signing anymore, and the new ZSK is being activated and used to sign the zone. The old key is still active to ensure the transition to the new key is smooth, and can validate any records leftover in the cache.

```
;; QUESTION SECTION:
;quad.jmu.lab.          IN A

;; ANSWER SECTION:
quad.jmu.lab.           86400 IN A 192.168.107.64
quad.jmu.lab.           86400 IN RRSIG A 8 3 86400 (
                                20260505094817 20260421172535 14329 jmu.lab.
                                WybuWtvdnhnTWEpaxqrNbCcQlr0xaZAmSVUnrdn7s5AD
                                Kp2jCuxk6W6IRIhS3KrHOKs1+aaMlt5X3Oyj9wvQgmTP
                                4j1R8Kpz1q/ACA9qjgFZLCmnWmM3uPybZ8kUb1jrA5dz
                                BTnzL/g4O+srZKBAXVbL/2BlCY7x1hUyFrZE6sI= )
quad.jmu.lab.           86400 IN RRSIG A 8 3 86400 (
                                20260505094817 20260421172535 49498 jmu.lab.
                                QNlRNEvoM55DIFI1EBkimu3OSwUbCZtGLuEoHQlUIOr7
                                suivZ3Io26cLX14lTyK8LbYLvyhY/tEuV71K5ArJlbI3
                                M7QHBM/13lnya0Se8GTynnoCzuMlXX3d4l0O5RheZhDm
                                SzAq+o0GyH33TPP5I1e4HkJiBu2mBfGvlJOUkQg= )
```

Figure 3.10: A dig query for quad.jmu.lab that shows two signatures attached to the address record. This is because the old ZSK is still active, and the new ZSK is being activated.

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.128 jmu.lab DNSKEY +dnssec +short
257 3 8 AwEAAaF2mbLD0Yr1NeFa69Q6GAlm/f4sdvJWrwMHZh5g/n8c+QqACrBc 94dr1VayDrFp6+XctI7htJm2weuaqRHrWysopYZwU9TA3rXjklXePmRA D04e4r3/iIwEF2vuP4pmNxQSmoVu
sqW1zLPtTa2EZFTa7EEpLr6fCFrp JC0LRY8jnnL90mbUStUSJdqISKOMm1hnEjtd5EvnGFsD9Vok9pBoqjGC yAoh1TLruOohXdzXHcavgl2rFX5b4RZQETeGDNK8FczzXtfKKXrDVhC/ f4J2V5i
ug5NTPb3mIWKLoeJMhoR4bMXjYJ3+BNezkpihO67K/rVseqpX kalbaHJpIHM=
256 3 8 AwEAAc3kJQXHUvzhk3HfbBgZ3I6NwDb0afzdSsIsqsDiJcjIH4sbLYPF pPJhBVkL61V5JWKlVqQYSpoX1/iu7lQwEHDKNtr4NhslarKU06TqJmHS 67UOtJxok19M3uNlRNZnmsIGb52i
IvKGUjHKcYdQkJozw+BuG6peI96G Kex6ny+F
256 3 8 AwEAAeTywLH5lSxJRaDsbLClrfsgEB/JH2xxUZDzqpiX7/+4NvBEBrjM WDEL2nh+aqOoW2F/sMucpu2mRt1zEZAPnDtvasJjm4+/2+ZONmiBGr9v sbArtBLXdOgpGyEhF+hhXJnqf91h
Xpa4MqHRHoVBJuijLcP12dA0OP9O bmVKVq6z
DNSKEY 8 2 3600 20260505182535 20260421172535 34939 jmu.lab. Z78EA9o8+OuGOA6pDNKdXLXS9JremfzkQ7aFAr1ntKpVY6IQI5EvgCo5 3HPWK+6slaYhJEVB4JXXIyByAEfyTcAF
3bSEoOTCOYAgKc4KdbPOOJhv kJAiu8ZGyOd/nobVc/Jz46pmFIPpXlKB44QYxU8+zRNPagJJz+w5yEHG SJlcpMsZ4E1S0i9dGDMk+stqBcMT1Z/oes1dZiGKBg2pYgDzWefRHI3w WWuYo7++w9m
Ax/cPN/hDKqOMchZCKeNhBqDi5p/Mqv9AuRwA6L8f7WQz m18mUvmpyJq6ieD+w4myDX5TVos7uMW5+CnmFz/740cVH00wCCyiDw8f cimCaw==
```

Figure 3.11: A dig query that has been shortened to show the two ZSKs (256), one inactive and one being activated and the KSK(257).

To fix this quickly for my own purposes and use the new keys, I was forced to manually

remove the old key, which would've been gone after a week per my setup settings. However, this is not how key rollovers are supposed to work, and there is a process for this. The process for key rollovers is as follows:[15]

- First, the new key is generated and added to the zone, and then signed with the old key. This way, the new key is trusted by the parent zone, and can be used to sign records.
- Then, the clock is ticking on the RRSIGs signed by the old key, because when they expire, the old key is removed from the zone. This way, there is no gap in trust, and the new key can be used without interruption.
- Finally, the old key is removed from the parent zone after the signatures expire, and the key rollover is complete.

This is what the keys would look like before you remove the old key that is still active but not being used to sign anymore, pulled for my .lab server, figure 3.12. The RRSIGs do not expire for two weeks, so this key will live that long.

On the jmu.lab server, I was a bit hasty in my actions and removed the old key assuming that inline signing would just fix the signatures and use the new key, but this is not how it should've been done. Instead, there was a gap in trust where the old key was removed before the new key was activated, which caused the signatures to fail validation. After removing the key, I used the status command to see how my setup was looking on this server, seen below in figure 3.13.

```
steve@steve-VMware-Virtual-Platform:/var/cache/bind$ sudo rndc dnssec -status lab.

dnssec-policy: rsa-policy
current time:  Fri Apr 24 16:33:57 2026

key: 34104 (RSASHA256), ZSK
  published:      yes - since Sun Feb 15 17:18:27 2026
  zone signing:   no

  Key is retired, will be removed on Sun Apr 26 19:23:27 2026
  - goal:           hidden
  - dnskey:         omnipresent
  - zone rrsig:     unretentive

key: 12978 (RSASHA256), ZSK
  published:      yes - since Tue Apr 21 14:17:08 2026
  zone signing:   yes - since Tue Apr 21 16:22:08 2026

  Next rollover scheduled on Sat Jun 20 14:17:08 2026
  - goal:           omnipresent
  - dnskey:         omnipresent
  - zone rrsig:     rumoured

key: 5852 (RSASHA256), KSK
  published:      yes - since Sun Feb 15 17:18:27 2026
  key signing:    yes - since Sun Feb 15 17:18:27 2026

  Next rollover scheduled on Mon Feb 15 15:13:27 2027
  - goal:           omnipresent
  - dnskey:         omnipresent
  - ds:             rumoured
  - key rrsig:      omnipresent
```

Figure 3.12: The first ZSK, 34104, is the old key that doesn't sign anymore, below is the new ZSK, 12978, that is being activated and signing the zone. The KSK, 5852, is still active and signing the ZSKs.

```
steve@steve-VMware-Virtual-Platform:~$ sudo rndc dnssec -status jmu.lab.
dnssec-policy: rsa-policy
current time:  Fri Apr 24 16:24:53 2026

key: 34939 (RSASHA256), KSK
  published:      yes - since Sun Feb 15 18:32:14 2026
  key signing:    yes - since Sun Feb 15 18:32:14 2026

  Next rollover scheduled on Mon Feb 15 16:27:14 2027
  - goal:           omnipresent
  - dnskey:         omnipresent
  - ds:             rumoured
  - key rrsig:      omnipresent

key: 49498 (RSASHA256), ZSK
  published:      yes - since Tue Apr 21 14:25:35 2026
  zone signing:   yes - since Tue Apr 21 16:30:35 2026

  Next rollover scheduled on Sat Jun 20 14:25:35 2026
  - goal:           omnipresent
  - dnskey:         omnipresent
  - zone rrsig:     rumoured
```

Figure 3.13: This figure shows the status of the keys on the jmu.lab server after the old ZSK was removed. The new ZSK is active and being used to sign records, but the old ZSK was deleted before the new key was activated, which caused a gap in trust and the signatures to fail validation.

3.5 DNSSEC Options

My client is the validator, with dnssec-validation set to yes. It has a trust anchor that was manually copied from the parent zone.

3.5.1 DNSSEC Policy

I adjusted the settings in the options configuration file, mainly to specify what algorithm to use, but also to set specifics for the keys. This is seen below in figure 3.14. Setting these settings also means that the keys are generated in the background with these specifications.

```
dnssec-policy rsa-policy {
        keys {
                ksk lifetime 365d algorithm RSASHA256 2048;
                zsk lifetime 60d algorithm RSASHA256 1024;
        };
        purge-keys 7d;
};
```

Figure 3.14: This is what I added to my dnssec policy I created called ”rsa-policy”. This was then added to the zone configuration to implement this policy.

I later added the purge-keys option to my policy, which automatically removes old keys after a certain period of time. This is a good option to have, because it ensures that old keys are removed in a timely manner, and that there is no gap in trust. This is especially important for the ZSK, which has a shorter TTL and is more likely to be compromised.

3.5.2 Signing

Right now, I am using Inline Signing, which automatically generates keys and signs the files. This setting also creates and keeps a separate signed zone file. Instead of running the `dnssec-signzone` and `dnssec-keygen` command, the server automatically signs the zone when it starts up. This makes resigning the zone when it expires on the server easier, and also makes it easier to manage the keys. The server will automatically generate the keys for the zone, and then sign the zone with those keys. I just specified the algorithm, size,

and time to live for the keys. It also reloads the zone automatically when the keys expire or zones are changed, which is a nice feature. This is a good option for this project, because it allows me to focus on the validation part of the project, rather than the signing part.

3.6 DNSSEC In The Testbed Environment

With DNSSEC on the internet, there is normally a validating resolver separate from the client machine that authenticates and validates the answers from the server before the client gets the response back. Normally, this is done recursively. For this study, there is no recursion and instead the answers will be manually validated.

Below, in figure 3.15, is an example of the Chain of Trust in the environment.

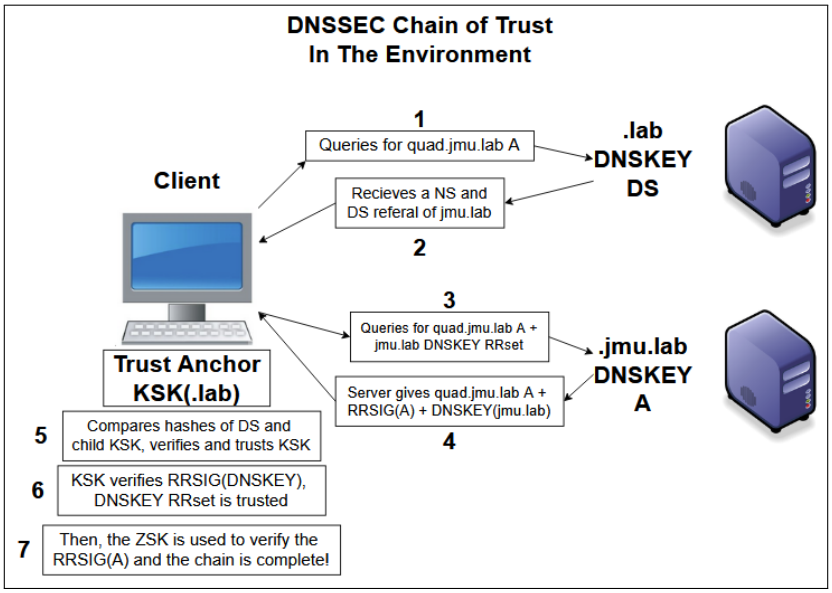


Figure 3.15: A diagram showing the Chain of Trust, with the trust anchor at the root, and the parent and child zones below. The arrows indicate the direction of trust, with each level trusting the next.

3.7 Simulated Attacks with DNSSEC Enabled

Building off of the previous attacks covered in DNS Attacks, I will now talk about how these attacks are different with DNSSEC enabled. Firstly, I should mention that I had to use manual signing to simulate these attacks, because with inline signing, the server automatically resigns the zone when it is reloaded. This means that if I were to change the db file to have a malicious zone, I had to run the `dnssec-signzone` command to resign the zone with the new changes, and then reload the zone to have the changes take effect and not get automatically signed. An example of this command is below in figure 3.16.

```
steve@steve-VMware-Virtual-Platform:/var/lib/bind$ sudo dnssec-signzone -K /var/
cache/bind -o lab. db.lab
Verifying the zone using the following algorithms:
- RSASHA256
Zone fully signed:
Algorithm: RSASHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                      ZSKs: 1 active, 0 stand-by, 0 revoked
db.lab.signed
```

Figure 3.16: A capture of the `dnssec-signzone` command being used to sign the lab zone.

Below is an example of a valid `delv` command, which is used to validate the answers from the server. This command is used to show that the answers are valid and have not been tampered with. In this query, `delv` gets the address record for computer.lab and the corresponding RRSIG, and `delv` validates this in the background, showing that the answer is valid and has not been tampered with, as seen in figure 3.17.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A
; fully validated
computer.lab.           86400   IN      A       192.168.107.2
computer.lab.           86400   IN      RRSIG   A 8 2 86400 20260412223532 20260
313223532 34104 lab. NWPQcfaC2Th/jhCe5P/QihLYrXB9+CZJ8az8H6bft5L/xWRmTOgPdn26 2r
smskBIMSnGCeMUTgX+jnS4ET5Rp7FJbDdefkTi984fvUG5t0rccvE8 zTw5+NVfC2jw0kVP1ZgYRwRTM
eq8bklL6ImYI4b02kypF2wxYyD4wFXU JWA=
```

Figure 3.17: The fully validated output when `delv` executes without error, showing the record and the accompanying signature.

After signing the data, I had to reload the zone to have the changes take effect. This is done with the `rndc reload` command, which updates the new configurations. So, to prove that

DNSSEC is working, I signed the zone, then made a change to the db file to have a malicious zone, and then reloaded the zone. After doing this, I ran a dig query for computer.lab, and got the following response, figure 3.18.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 jmu.lab NS +rtrace
;; fetch: jmu.lab/NS
;; DNS format error from 192.168.107.129#53 resolving jmu.lab/NS for <unknown>:
non-improving referral
;; FORMERR resolving 'jmu.lab/NS/IN': 192.168.107.129#53
;; resolution failed: failure
```

Figure 3.18: This is the output of a `delv` command that caught a malicious injection and rejected the response.

3.7.1 Corrupting the DNS Data

To test validation with DNSSEC, I wanted to corrupt records, like the signatures and Trust Anchors to see what would happen. Below, in figure 3.19, is an example of deleting a single character from the RRSIG record, which is the signature that is used to validate the answer. As you can see, the RRSIG is not valid, because the data was tampered with, and the answer is not authenticated. This shows that DNSSEC is working, because it is able to detect that the data was tampered with and is not valid.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A +rtrace
;; fetch: computer.lab/A
;; fetch: lab/DNSKEY
;; validating computer.lab/A: no valid signature found
;; RRSIG failed to verify resolving 'computer.lab/A/IN': 192.168.107.129#53
;; resolution failed: RRSIG failed to verify
```

Figure 3.19: The output above shows a `delv` command for computer.lab, but the RRSIG is not valid, because the data was tampered with.

Testing just a corrupt signature was not enough, so I also wanted to test what would happen if the trust anchor was corrupted. Below, in figure 3.20, is an example of deleting a single character from the trust anchor, which is the root of the chain of trust. With a corrupted trust anchor, the client cannot verify the parent zone's KSK, which means that the client cannot verify any of the records from the parent zone or any of the child zones.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab. @192.168
.107.129 computer.lab A
;; validating lab/DNSKEY: no valid signature found (DS)
;; no valid RRSIG resolving 'lab/DNSKEY/IN': 192.168.107.129#53
;; broken trust chain resolving 'computer.lab/A/IN': 192.168.107.129#53
;; resolution failed: broken trust chain
```

Figure 3.20: The error above shows a `delv` command for computer.lab, but the trust anchor is not valid, because the data was tampered with.

Algorithms and Data Structures For DNSSEC

4.1 An Introduction to DNSSEC Signing

Earlier, different records were explained and new resource records were introduced, but how do they actually get signed? Before signing can occur, keys must be generated with `dnssec-keygen`. This tool creates the necessary public and private keys that will be used for signing the zone file. After the keys are generated, the signing process can be manually completed using a tool called `dnssec-signzone`. Running this tool in the command line will create the necessary signatures for the zone file.

The data is built according to how it is laid out previously in Resource Record Signature. In my implementation, I read in a file with the hexadecimal representation of the data, pulled from Wireshark captures.

4.2 Cipher Suites

4.2.1 Hashing with SHA256

The algorithm of choice for hashing in this project is SHA-256. This algorithm is widely used in many cryptographic applications, and is considered to be a secure hashing algorithm. It produces a 256-bit hash, which is a fixed size output that is unique to the input data. This means that even a small change in the input data will produce a completely different hash,

making it difficult for attackers to create a hash collision. Having 2^{256} possible combinations of hashes makes it infeasible for attackers to brute force their way to a collision, which is when two different inputs produce the same hash. With this collision resistance, SHA-256 is a good choice for hashing in DNSSEC, as it helps to ensure the integrity of the data being signed and verified.

Seen below is two slightly different strings being hashed by SHA256, their outputs seen in figure 4.1.

```
dormadsa@stu:~$ echo -n "hello" | openssl dgst -sha256
SHA2-256(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
dormadsa@stu:~$ echo -n "Hello" | openssl dgst -sha256
SHA2-256(stdin)= 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Figure 4.1: Two strings, ”hello” and ”Hello” being hashed, producing two very different digests.

4.2.2 Alternate Hashing Algorithms

SHA256 is not the only option in DNSSEC for hashing the meta data. There are various other functions, like MD5, SHA1, SHA384, SHA512. There are various pros and cons for each hashing algorithm. For instance with SHA512, compared to the 2^{256} of SHA256, 2^{512} is an astronomically larger number of possible hashes, but also doubles the length overall. RFC8624 states that hashing with SHA 512 is NOT RECOMMENDED, due to the increase in message size and limited added security. However, you must be able to validate with SHA512, because it is available to be used. [16]

4.2.3 Signing with RSA

My algorithm of choosing for the signing half of the signatures is RSA. The two keys, the KSK being 2048 bits and the ZSK being 1024 bits, differe in size because the KSK is important and gets used less frequently than the ZSK. From using OpenSSL to creating structs to store data, there were various algorithms and data structures that were used to

accomplish this task. RSA is a solid option for keys as of now, they are still very difficult and infeasible to crack, especially with a bigger key size. However, with the rise of quantum computing, changes may have to be made.

In the environment, I am manually signing these records with the private key pulled straight from the server myself, and then using the public key to verify the signatures. This is a good way to test the validation process, because I can control the data that is being signed and verified. Not only that, but I am also creating the signature myself to prove that the received signature is valid as well.

4.2.4 Alternate Signing Algorithms

Some alternative algorithms that could be used for signing are DSA (Digital Signature Algorithm), ECDSA (Elliptic Curve Digital Signature Algorithm), Ed25519. However, ECDSA provides the same if not better security as RSA with smaller key size. ECDSA signatures are much smaller than RSA signature, roughly half to a quarter of the size in some cases. This algorithm is also generally faster when signing, but maybe slower verifying compared to RSA. The math is also very complex with this algorithm, compared to the more robust RSA.[17] A great site showing more inofrmation about the comparison between DNSSEC algorithms is here. More information on both hashing and signing algorithms and their compatibility with DNSSEC can be found at RFC8624.

4.3 RRSIG Construction

To build the RRSIG records, I followed the format laid out in Resource Record Signature. I read in the data from a file, which was pulled from Wireshark captures, and then built the RRSIG record according to the format. First, the RRSIG metadata is built, which includes the type of record, the algorithm used, the labels, the original TTL, the signature expiration and inception times, the key tag, and the signer's name. Then, the canonicalized RRSet is

concatenated to the data, and the whole thing is hashed using SHA-256. Finally, the hash is signed using the RSA algorithm with the private key, and the signature is added to the RRSIG record. Pictured below in figure 4.2 is the general layout for the data to be hashed. In this image, the record type is an A record, and the rest of the data is filled in with the appropriate values for the record being signed.

Field	Size	Description
Type Covered	2 bytes	RR type being signed (A=0x0001)
Algorithm	1 byte	8 = RSASHA256
Labels	1 byte	Number of labels in owner name
Original TTL	4 bytes	TTL from the RRSIG record
Sig Expiration	4 bytes	Unix timestamp, big endian
Sig Inception	4 bytes	Unix timestamp, big endian
Key Tag	2 bytes	Key tag of signing ZSK
Signer's Name	variable	Zone name in DNS wire format
Owner Name	variable	RR owner name in DNS wire format
Type	2 bytes	Same as type covered
Class	2 bytes	IN = 0x0001
TTL	4 bytes	Must match original TTL exactly
RDLENGTH	2 bytes	Length of RDATA
RDATA	variable	The actual record data (e.g. IP address)

Figure 4.2: A diagram of how the data is laid out and then hashed for signatures, generated by Claude.

4.4 OpenSSL

For this project, I used OpenSSL behind the scenes with the tools I ran in the previous section. OpenSSL is a powerful library that provides various cryptographic functions, including those needed for DNSSEC signing. It allows us to generate keys, create signatures, and manage certificates, which are essential components of the DNSSEC infrastructure. OpenSSL even provides support for converting data from Base64 into binary

When I run the `dnssec-signzone` command, it utilizes OpenSSL to perform the cryptographic operations required for signing the zone file. By doing this, it hashes the data and

then signs it with the private key in the background.

To complete the signing process, OpenSSL provides a function called EVP_DigestSign, which allows us to create a signature using the specified hashing algorithm and private key. This function takes care of the details of the signing process, making it easier for us to implement DNSSEC signing in our code. Rather than having to separately hash and then sign the data, this function takes care of both.[18]

Programming the Signature and Validation of Several DNSSEC Resource Records

5.1 Main Functionality

There are several arguments that can be passed to the program:

1. The first argument is the name of the file that contains the private key that will be used for signing.

2. The second argument is the name of the file that contains the public key that will be used for verifying the signature.

3. The third argument is the name of the file that contains the hexadecimal representation of the data to be signed.

4. The final argument is the name of the file that will contain the signature from the server in Base64 format.

This list of inputs is seen below in the code and comments in figure 5.1. This is an early catch to prevent any other errors from occurring down the line, while also ensuring a clean input is received.

```
// ------------------------------------------------------
// Argument validation
// The program needs exactly 4 arguments:
//   1. private.pem  - private key for signing
//   2. public.pem   - public key for verification
//   3. sigbase.hex  - the signature base we constructed
//   4. rrsig.b64    - the real signature from the zone
// ------------------------------------------------------
fprintf( stdout , "\n=============================================\n");
if ( argc != 5 )
{
    fprintf( stderr , "Usage: %s <private.pem> <public.pem> <sigbase.hex> <rrsig.b64>\n\n" , argv [ 0 ]  );
    fprintf( stderr , "  private.pem  - private key in PEM format\n" );
    fprintf( stderr , "  public.pem   - public key in PEM format\n" );
    fprintf( stderr , "  sigbase.hex  - signature base as a hex string file\n" );
    fprintf( stderr , "  rrsig.b64    - existing RRSIG signature field as base64 file\n" );
    return EXIT_FAILURE;
}
```

Figure 5.1: A snippet of code at the top of the main function, validating the input and providing a helpful message to make sure the user inputs the correct elements in the correct order.

Using these arguments, the data is read into buffers in the program, and the base64 signature is read in, converted to binary, and stored in a buffer as well. Firstly, a signature is generated using OpenSSL's EVP_DigestSign family of functions, seen below in Signing the Data section, figure 5.5. The signature base, or hexadecimal notation of the data to be hashed, is passed in here and stored as the generated signature. This signature is then passed to another family of OpenSSL functions, EVP_DigestVerify, seen in Verifying the Signature, figure 5.6 which uses the public key to cryptographically verify the signature. The same happens with the existing signature that was queried from the server, seen in figure 5.7. After these verifications, both of these signatures are passed to a helper function that compares the signature byte by byte, printing them to the terminal in a hexdump to easily see any differences, and outputting whether they were a match or not. Below, in figure 5.2, is an outline of what the code does

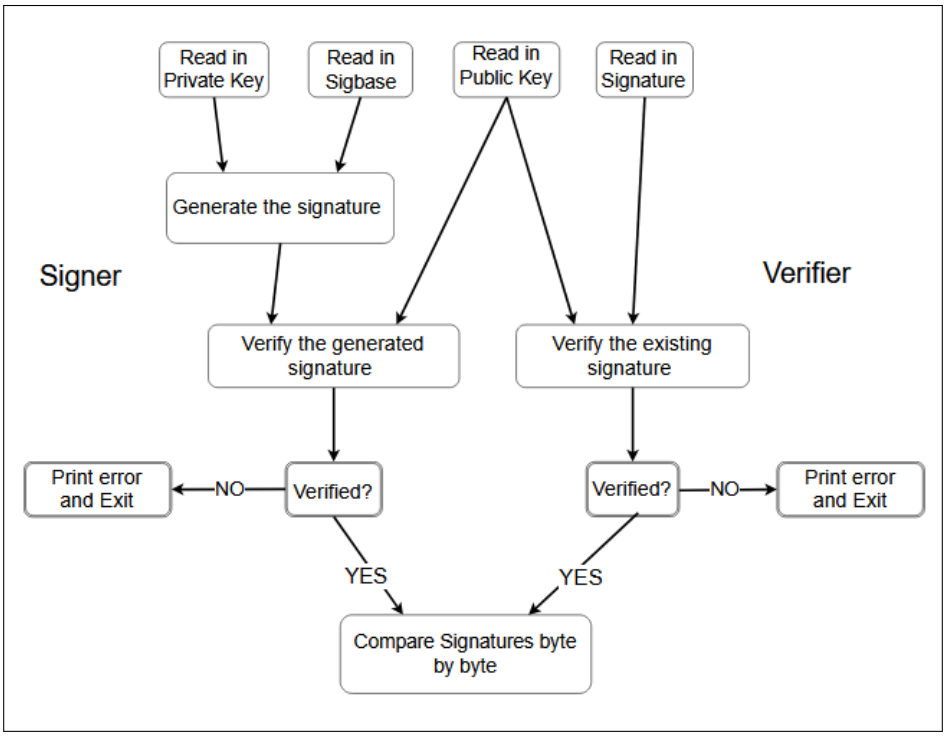


Figure 5.2: A diagram showing an overview of the flow of the code that I wrote for signing and validating signatures.

5.2 Converting Base64 Data

To convert the Base64 signature from the file as well as the public and private keys into a format that can be used for verification, I used OpenSSL's Base64 decoding functions. This allows me to take the Base64-encoded signature and convert it back into its original binary format, which is necessary for the verification process. The code for this conversion is shown in figure 5.3. I enlisted Anthropic's Claude Code Interpreter to help me with this part, and it was able to generate the code for me based on the OpenSSL documentation. This allowed me to focus on the actual signing and validation portions of the project, while these these utility functions were implemented with the assistance of Claude.

```
// ========================================
// Decode a base64 string into bytes ( handles newlines/whitespace )
// Written by Claude, adapted from OpenSSL BIO base64 decoding examples
// Example: "AAEC" -> { 0x00, 0x01, 0x02 }
// ========================================
int b64_to_bin( const char *b64 , unsigned char **out , size_t *out_len )
{
    // Strip whitespace into a clean buffer
    char *clean = malloc( strlen( b64 ) + 1 );
    int ci = 0;
    for ( int i = 0; b64 [ i ] ; i++ )
    {
        if (   b64 [ i ]  != ' ' && b64 [ i ]  != '\n'
            && b64 [ i ]  != '\r' && b64 [ i ]  != '\t' )
            {
                clean [ ci++ ]  = b64 [ i ] ;
            }
    }
    clean [ ci ]  = 0;

    size_t bin_len = ( ci * 3 ) / 4 + 4;
    *out = malloc( bin_len );

    BIO *b64_bio = BIO_new( BIO_f_base64 () );
    BIO *mem_bio = BIO_new_mem_buf( clean , -1 );
    BIO_push( b64_bio , mem_bio );
    BIO_set_flags( b64_bio , BIO_FLAGS_BASE64_NO_NL );

    int decoded = BIO_read( b64_bio , *out , bin_len );
    BIO_free_all( b64_bio );
    free( clean );

    if ( decoded <= 0 )
    {
        fprintf( stderr , "Error: base64 decode failed\n" );
        free( *out );
        return 0;
    }

    *out_len = decoded;
    return 1;
}
```

Figure 5.3: Code for converting the Base64-encoded signature from the file into its original binary format using OpenSSL's Base64 decoding functions, written by Claude.

5.3 Alternative Key Encoding

The keys that are stored on the DNSSEC servers are in a format that is not directly usable in the C code. Therefore, I had to convert them from the DNSSEC format to PEM format, which is a widely used format for storing cryptographic keys. I had Anthropic's Claude help me with this conversion and wrote code to convert the keys. This code takes both a public

and private key from the .key and .private files from the server, and converts the keys into a .pem RSA format that can be used in the C code for signing and verifying the signatures. Overall, the code reads in the DNSSEC key data, determines if it is a public or private key, and then uses OpenSSL functions to convert the key data into the appropriate PEM format. The way the keys are stored in the .key and .private files is in a format that includes extra information, such as the key tag and algorithm, which is not needed for the PEM format.

5.4 Hashing the Data

To hash the data, I used OpenSSL's SHA256 algorithm. This algorithm is widely used in many cryptographic applications, and is considered to be a secure hashing algorithm. It produces a 256-bit hash, which is a fixed size output that is unique to the input data. This means that even a small change in the input data will produce a completely different hash, making it difficult for attackers to create a hash collision. The code that just hashes the data is shown in figure 5.4.

```
// == Compute SHA-256 digest ==
unsigned char digest [ 32 ] ;
EVP_MD_CTX *ctx = EVP_MD_CTX_new ();
EVP_DigestInit_ex( ctx , HASH_ALGORITHM , NULL );
EVP_DigestUpdate( ctx , data , len );

unsigned int digest_len;
EVP_DigestFinal_ex( ctx , digest , &digest_len );
EVP_MD_CTX_free( ctx );
```

Figure 5.4: Code from a helper function that hashes the data using OpenSSL's SHA256 algorithm, used for printing the hash of the data to be signed.

5.5 Signing the Data

For signing the hashed data, RSA keys were used. For signing, only the private keys were needed. The private key is used to sign the data, and the verification code is shown in Verifying the Signature. The code that just signs the data is shown in figure 5.5.[18]

```
// ==============================
// Generate the signature
// ==============================
unsigned char *gen_sig = NULL;
size_t gen_sig_len = 0;

// Sign the sigbase with the private key using SHA-256 and the private key
EVP_MD_CTX *sign_ctx = EVP_MD_CTX_new ();
if( !sign_ctx )
{
    fprintf( stderr , "Error: failed to create signing context\n" );
    return EXIT_FAILURE;
}

if (   EVP_DigestSignInit  ( sign_ctx , NULL , HASH_ALGORITHM , NULL , privkey ) != 1
    || EVP_DigestSignUpdate( sign_ctx , sigbase , sigbase_len ) != 1
    || EVP_DigestSignFinal ( sign_ctx , NULL , &gen_sig_len ) != 1 )
{
    fprintf( stderr , "Error: signing init failed\n" );
    return EXIT_FAILURE;
}

gen_sig = malloc( gen_sig_len );

// Do final signing step to get the actual signature bytes.
// This is where the RSA operation happens.
if ( EVP_DigestSignFinal( sign_ctx , gen_sig , &gen_sig_len ) != 1 )
{
    fprintf( stderr , "Error: signing final failed\n" );
    return EXIT_FAILURE;
}
// ==============================
// Signature generated
// ==============================
```

Figure 5.5: This code is showing how the signature is created using OpenSSL's RSA signing functions. I am assuming the role of the DNS Server in this code, using the private key to sign the data.

5.6 Verifying the Signature

The code in figure 5.6 and figure 5.7 shows how the signature is verified using OpenSSL's RSA verification functions, the EVP_DigestVerify family. I am assuming the role of the DNS Client in this code, using the public key to verify the signatures. First, I verify the

generated signature, which is the signature that I created in the previous section. Then, I verify the received signature, which is the signature that was read in from the file that was converted from Base64. If both of these return without error, both signature are then passed to the helper function compare_sigs.[19]

```
// ==============================
// Verify generated Signature with public key
// ==============================
printf( "=== Verify: generated signature with public key ===\n" );
EVP_MD_CTX *verify_ctx = EVP_MD_CTX_new ();
if( !verify_ctx )
{
    fprintf( stderr , "Error: failed to create verification context\n" );
    return EXIT_FAILURE;
}

if (   EVP_DigestVerifyInit  ( verify_ctx , NULL , HASH_ALGORITHM , NULL , pubkey ) == 1
    && EVP_DigestVerifyUpdate( verify_ctx , sigbase , sigbase_len ) == 1
    && EVP_DigestVerifyFinal ( verify_ctx , gen_sig , gen_sig_len ) == 1 )
{
    printf( "PASS: generated signature verifies correctly\n\n" );
}
else
{
    printf( "FAIL: generated signature did not verify\n\n" );
}
EVP_MD_CTX_free( verify_ctx );
// ==============================
// Generated Signature has been verified (or not...)
// ==============================
```

Figure 5.6: Code for verifying the signature using OpenSSL's RSA verification functions.

```
// ==============================
// Verify existing Signature with public key
// ==============================
printf( "=== Verify: existing signature with public key ===\n" );
verify_ctx = EVP_MD_CTX_new ();
if( !verify_ctx )
{
    fprintf( stderr , "Error: failed to create verification context\n" );
    return EXIT_FAILURE;
}

if (   EVP_DigestVerifyInit  ( verify_ctx , NULL , HASH_ALGORITHM , NULL , pubkey ) == 1
    && EVP_DigestVerifyUpdate( verify_ctx , sigbase , sigbase_len ) == 1
    && EVP_DigestVerifyFinal ( verify_ctx , existing_sig , existing_sig_len ) == 1 )
{
    printf( "PASS: existing signature verifies correctly\n\n" );
}
else
{
    printf( "FAIL: existing signature did not verify - sigbase may not match exactly\n\n" );
}

EVP_MD_CTX_free( verify_ctx );
// ==============================
// Exisiting Signature has been verified (or not...)
// ==============================
```

Figure 5.7: Code for verifying the signature using OpenSSL's RSA verification functions.

5.7 Generating and Validating an Address Resource Record Signature

The Address Resource Record Signature is the most basic signature. This signature consists of the RRSIG meta data, describing what record is being signed and the logistics of the signature, like TTL and an expiration date, etc. Appended to the header, is the owner name (e.g. computer.lab.), type, class, TTL, RDLENGTH, specifying the length of the RDATA payload, in this case 4 for IPv4, then four bytes four the IP. For computer.lab, it would look like this: 192.168.107.2 becomes c0 a8 6b 02. As seen before, in figure 4.2, this shows an Address records data. Below, in figure 5.8, is the output from my code, showing the matching signatures.

```
=== Record Type: A (Address Record) ===

=== Existing RRSIG signature ( 128 bytes ) ===
0000  aa aa ce b3 34 13 31 67  5b e0 a0 d2 31 d1 ce e8  |....4.1g[...1...|
0010  d7 ab 01 4d ef 55 59 b5  40 4e 74 8f d5 c5 f5 5d  |...M.UY.@Nt....]|
0020  88 c5 9e 0d 76 1f 3c 0b  91 75 36 13 b7 41 fb c0  |....v.<..u6..A..|
0030  37 a4 04 4e 07 17 e5 68  7b 56 27 16 35 14 dd d7  |7..N...h{V'.5...|
0040  37 1d 54 08 18 a9 e2 6c  25 db 32 8b ee a2 96 86  |7.T....l%.2.....|
0050  88 68 e9 99 85 31 ca 9c  d1 36 58 4e 2e c6 49 f5  |.h...1...6XN..I.|
0060  ae a5 c2 17 92 a7 64 85  0a 4c ce 53 57 68 97 16  |......d..L.SWh..|
0070  9a 1e 77 b0 55 43 b8 89  fe 6f c0 9f 1c 6b f0 0f  |..w.UC...o...k..|


=== Generated signature ( 128 bytes ) ===
0000  aa aa ce b3 34 13 31 67  5b e0 a0 d2 31 d1 ce e8  |....4.1g[...1...|
0010  d7 ab 01 4d ef 55 59 b5  40 4e 74 8f d5 c5 f5 5d  |...M.UY.@Nt....]|
0020  88 c5 9e 0d 76 1f 3c 0b  91 75 36 13 b7 41 fb c0  |....v.<..u6..A..|
0030  37 a4 04 4e 07 17 e5 68  7b 56 27 16 35 14 dd d7  |7..N...h{V'.5...|
0040  37 1d 54 08 18 a9 e2 6c  25 db 32 8b ee a2 96 86  |7.T....l%.2.....|
0050  88 68 e9 99 85 31 ca 9c  d1 36 58 4e 2e c6 49 f5  |.h...1...6XN..I.|
0060  ae a5 c2 17 92 a7 64 85  0a 4c ce 53 57 68 97 16  |......d..L.SWh..|
0070  9a 1e 77 b0 55 43 b8 89  fe 6f c0 9f 1c 6b f0 0f  |..w.UC...o...k..|

=== Verify: generated signature with public key ===
PASS: generated signature verifies correctly

=== Verify: existing signature with public key ===
PASS: existing signature verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
================================================
```

Figure 5.8: Output from validating an A RRSIG record.

5.8 Generating and Validating a Name Server Resource Record Signature

The Name Server Resource Record was a bit more tricky than the Address Record Signature. After the signature header, once again the owner name follows, (e.g. jmu.lab.), then the type (0x0002), class IN (0x0001), the TTL, the RDLENGTH and the RDATA. For the jmu.lab server, the length of the RDATA would be 12 bytes, or 0x000c in hex, and the RDATA is ns.jmu.lab., which becomes 02 6e73 03 6a6d75 03 6c6162 00 in hex.

```
=== Record Type: NS (Name Server Record) ===

=== Existing RRSIG signature ( 128 bytes ) ===
0000  61 59 93 11 78 b6 d2 58  54 38 a5 86 50 96 57 0e  |aY..x..XT8..P.W.|
0010  59 fb 9d b8 82 6b b0 91  17 5d 97 1b 1b 17 90 45  |Y....k...]....E|
0020  1f 05 58 17 dc ad 49 39  6b 31 6b 82 90 0b 74 24  |..X...I9k1k...t$|
0030  88 27 89 17 21 e4 15 fe  dc 8b c6 ed 6b b9 66 e8  |.'..!.......k.f.|
0040  e9 9c e2 2c bd 9f d9 0c  0e bb 65 24 08 4e 99 67  |...,......e$.N.g|
0050  42 f7 89 31 89 ca 64 e2  8a 20 2c 3f ce 91 16 e1  |B..1..d.. ,?....|
0060  6b a2 2c 94 84 ca 81 78  55 83 5c 2c 17 5a 8b 61  |k.,....xU.\,.Z.a|
0070  a8 93 6e 6c 3e 99 99 ae  75 fe f4 e7 01 fb 07 38  |..nl>...u......8|


=== Generated signature ( 128 bytes ) ===
0000  61 59 93 11 78 b6 d2 58  54 38 a5 86 50 96 57 0e  |aY..x..XT8..P.W.|
0010  59 fb 9d b8 82 6b b0 91  17 5d 97 1b 1b 17 90 45  |Y....k...]....E|
0020  1f 05 58 17 dc ad 49 39  6b 31 6b 82 90 0b 74 24  |..X...I9k1k...t$|
0030  88 27 89 17 21 e4 15 fe  dc 8b c6 ed 6b b9 66 e8  |.'..!.......k.f.|
0040  e9 9c e2 2c bd 9f d9 0c  0e bb 65 24 08 4e 99 67  |...,......e$.N.g|
0050  42 f7 89 31 89 ca 64 e2  8a 20 2c 3f ce 91 16 e1  |B..1..d.. ,?....|
0060  6b a2 2c 94 84 ca 81 78  55 83 5c 2c 17 5a 8b 61  |k.,....xU.\,.Z.a|
0070  a8 93 6e 6c 3e 99 99 ae  75 fe f4 e7 01 fb 07 38  |..nl>...u......8|

=== Verify: generated signature with public key ===
PASS: generated signature verifies correctly

=== Verify: existing signature with public key ===
PASS: existing signature verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
================================================
```

Figure 5.9: Output from validating an NS RRSIG record.

5.9 Generating and Validating a DNSKEY Resource Record Signature

The DNSKEY Resource Record Signature is the biggest and one of the more difficult records to sign. This signature requires the KSK to be used, as the DNSKEYs are being signed, which entail means that these keys are to be passed to the program. The KSK is double the size of the ZSKs, and this leads to a doubly large signature. The output of this record is seen below in figure 5.10. The signature length expands to 256 bytes long from the previous 128 byte zone records.

Following the meta data of the RRSIG, the DNSKEY record has some unique data appended to it. It starts the same as the other records, with the owner name, type (0x0030 in this instance), class, and TTL of the record. Then the RDLENGTH is passed, and it is a combination of the two byte flags (0x0101 for KSK [257] and 0x0100 for ZSK [256]), single byte protocol, and single byte algorithm, followed by the raw key bytes of the public key, decoded from base64. Both the public KSK and ZSK are passed, so they must be canonically ordered. Using Wireshark, they were in canonical order from the traffic capture.

```
=== Record Type: DNSKEY (DNS Key Record) ===

=== Existing RRSIG signature ( 256 bytes ) ===
0000  80 ff f9 5b d7 b8 d8 2e  8e 56 46 28 b6 f2 d2 a3  |...[.....VF(....|
0010  7f d1 97 e9 7e 1c b8 5b  12 00 ca 35 bc f3 61 ee  |....~..[...5..a.|
0020  16 d6 85 63 d0 d3 55 91  23 de 3a 0a a8 dd 25 a6  |...c..U.#.:...%.|
0030  28 68 4f fb b2 92 4d 24  a0 0d f3 69 16 4c d7 44  |(hO...M$...i.L.D|
0040  7b 57 0b e2 87 56 30 b5  55 11 5c df f9 24 17 94  |{W...V0.U.\..$..|
0050  09 77 a0 4a e0 ab d5 62  25 e9 e4 f0 23 cc d2 25  |.w.J...b%...#..%|
0060  e5 29 4b 71 fe 97 26 a6  68 76 5c 98 1b f3 a7 2d  |.)Kq..&.hv\....-|
0070  0d 76 6a a0 4a b3 0f 5f  ae 2d 2d 5c c4 42 83 64  |.vj.J.._.--\.B.d|
0080  16 be 67 f5 05 ae 67 3a  fa 9d 8f 85 c8 39 02 45  |..g...g:.....9.E|
0090  0c ac df e4 3f 0f fc 21  4d 35 39 c1 18 4a fe 42  |....?..!M59..J.B|
00a0  e6 8c 06 41 92 46 23 9a  1c 75 90 1c 34 2d 5b 95  |...A.F#..u..4-[.|
00b0  b2 a4 c0 d3 5a e4 dd 19  4b 33 96 65 25 09 0d 32  |....Z...K3.e%..2|
00c0  19 94 7f d8 46 75 f1 2a  ef 6c c2 b7 0e 58 b0 4d  |....Fu.*.l...X.M|
00d0  63 e9 8c 49 41 1d a6 a8  f5 06 a4 c8 bd 2d c6 5d  |c..IA........-.]|
00e0  d2 8e 90 94 94 fa 43 94  e8 ab a0 1c 35 64 89 99  |......C.....5d..|
00f0  4e 58 65 25 57 5f fa 15  ca d5 f9 c0 18 7d fd 10  |NXe%W_.......}..|


=== Generated signature ( 256 bytes ) ===
0000  80 ff f9 5b d7 b8 d8 2e  8e 56 46 28 b6 f2 d2 a3  |...[.....VF(....|
0010  7f d1 97 e9 7e 1c b8 5b  12 00 ca 35 bc f3 61 ee  |....~..[...5..a.|
0020  16 d6 85 63 d0 d3 55 91  23 de 3a 0a a8 dd 25 a6  |...c..U.#.:...%.|
0030  28 68 4f fb b2 92 4d 24  a0 0d f3 69 16 4c d7 44  |(hO...M$...i.L.D|
0040  7b 57 0b e2 87 56 30 b5  55 11 5c df f9 24 17 94  |{W...V0.U.\..$..|
0050  09 77 a0 4a e0 ab d5 62  25 e9 e4 f0 23 cc d2 25  |.w.J...b%...#..%|
0060  e5 29 4b 71 fe 97 26 a6  68 76 5c 98 1b f3 a7 2d  |.)Kq..&.hv\....-|
0070  0d 76 6a a0 4a b3 0f 5f  ae 2d 2d 5c c4 42 83 64  |.vj.J.._.--\.B.d|
0080  16 be 67 f5 05 ae 67 3a  fa 9d 8f 85 c8 39 02 45  |..g...g:.....9.E|
0090  0c ac df e4 3f 0f fc 21  4d 35 39 c1 18 4a fe 42  |....?..!M59..J.B|
00a0  e6 8c 06 41 92 46 23 9a  1c 75 90 1c 34 2d 5b 95  |...A.F#..u..4-[.|
00b0  b2 a4 c0 d3 5a e4 dd 19  4b 33 96 65 25 09 0d 32  |....Z...K3.e%..2|
00c0  19 94 7f d8 46 75 f1 2a  ef 6c c2 b7 0e 58 b0 4d  |....Fu.*.l...X.M|
00d0  63 e9 8c 49 41 1d a6 a8  f5 06 a4 c8 bd 2d c6 5d  |c..IA........-.]|
00e0  d2 8e 90 94 94 fa 43 94  e8 ab a0 1c 35 64 89 99  |......C.....5d..|
00f0  4e 58 65 25 57 5f fa 15  ca d5 f9 c0 18 7d fd 10  |NXe%W_.......}..|

=== Verify: generated signature with public key ===
PASS: generated signature verifies correctly

=== Verify: existing signature with public key ===
PASS: existing signature verifies correctly

=== Binary Comparison ===
Generated signature length : 256 bytes
Existing  signature length : 256 bytes
MATCH: signatures are byte-for-byte identical
================================================
```

Figure 5.10: Output from validating an DNSKEY RRSIG record.

5.10 Generating and Validating a Start of Authority Resource Record Signature

The Start of Authority Resource Record signature was one on the most difficult to construct. It started off the same as the others, with the owner name, type (0x0006), class, and TTL.

The RDLENGTH is variable, and depends on the length of both the MNAME and RNAME. The MNAME is the primary nameserver, and the RNAME is the responsible mailbox, seen below in figure 5.11. Following those, there are five fixed length fields, which can be seen too in figure 5.11 below.

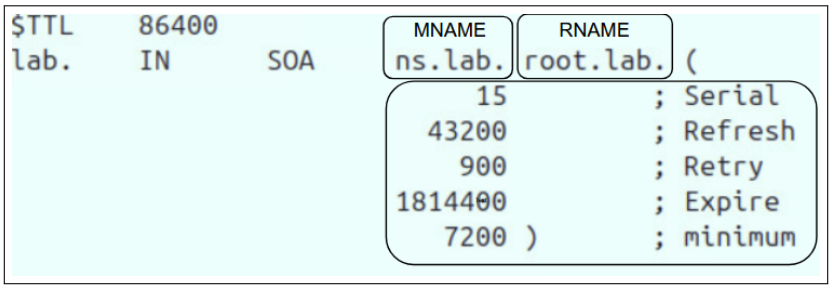


Figure 5.11: An example of an SOA record for my lab zone, showing the RNAME and MNAME, as well as the other fields.

All of these records are four bytes long, starting with the serial number, refresh, retry, expire, and minimum fields. The hardest part for me was getting the correct MNAME and RNAME values and having them in fully expanded wire format, then adding them to the RDLENGTH. Getting the RDLENGTH wrong by even one byte results in a completely wrong value. The output from my code when generating and validating an SOA signature is seen in figure 5.12 below.

```
=== Record Type: SOA (Start of Authority Record) ===

=== Existing RRSIG signature ( 128 bytes ) ===
0000  69 43 17 0c b8 a2 29 2f  26 77 16 f8 8f 63 63 f1  |iC....)/&w...cc.|
0010  20 da a2 08 a4 29 93 6b  02 7b b9 0f 5b 80 ed 78  | ....).k.{..[..x|
0020  b2 89 ed 71 42 48 26 b0  7e 25 6e b4 bb 3a 38 81  |...qBH&.~%n..:8.|
0030  88 29 c6 87 70 23 3e a2  84 9c 1c 19 a9 1f bb 34  |.)..p#>........4|
0040  d5 e6 2f 89 bc eb 35 44  94 13 e0 f1 05 98 31 c0  |../...5D......1.|
0050  9e d0 c6 0a 91 6c c3 f4  68 59 e5 74 77 45 99 d6  |.....l..hY.twE..|
0060  66 9c 29 a5 9e 85 23 39  ba d2 64 c0 fd 35 99 22  |f.)...#9..d..5."|
0070  91 7f fa 78 96 c1 99 69  73 7b b6 a6 27 b8 9a a1  |...x...is{..'...|


=== Generated signature ( 128 bytes ) ===
0000  69 43 17 0c b8 a2 29 2f  26 77 16 f8 8f 63 63 f1  |iC....)/&w...cc.|
0010  20 da a2 08 a4 29 93 6b  02 7b b9 0f 5b 80 ed 78  | ....).k.{..[..x|
0020  b2 89 ed 71 42 48 26 b0  7e 25 6e b4 bb 3a 38 81  |...qBH&.~%n..:8.|
0030  88 29 c6 87 70 23 3e a2  84 9c 1c 19 a9 1f bb 34  |.)..p#>........4|
0040  d5 e6 2f 89 bc eb 35 44  94 13 e0 f1 05 98 31 c0  |../...5D......1.|
0050  9e d0 c6 0a 91 6c c3 f4  68 59 e5 74 77 45 99 d6  |.....l..hY.twE..|
0060  66 9c 29 a5 9e 85 23 39  ba d2 64 c0 fd 35 99 22  |f.)...#9..d..5."|
0070  91 7f fa 78 96 c1 99 69  73 7b b6 a6 27 b8 9a a1  |...x...is{..'...|

=== Verify: generated signature with public key ===
PASS: generated signature verifies correctly

=== Verify: existing signature with public key ===
PASS: existing signature verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
================================================
```

Figure 5.12: Output from validating an SOA RRSIG record.

Bibliography

[1] BIND9, "Introduction to dns and bind 9." https://bind9.readthedocs.io/en/v9.18.39/chapter1.html#the-domain-name-system-dns, 2023. Accessed: 2026-1-1.

[2] Cloudflare Learning, "What is dns?." https://www.cloudflare.com/learning/dns/what-is-dns/, 2025. Accessed: 2025-12-22.

[3] P. Mockapetris, "Domain names - implementation and specification." https://www.rfc-editor.org/rfc/rfc1035, 1987. Accessed: 2026-1-1.

[4] Cloudflare Learning, "Dns a record." https://www.cloudflare.com/learning/dns/dns-records/dns-a-record/, 2025. Accessed: 2026-1-20.

[5] Cloudflare Learning, "Dns ns record." https://www.cloudflare.com/learning/dns/dns-records/dns-ns-record/, 2025. Accessed: 2026-1-20.

[6] Cloudflare Learning, "What is dns cache poisoning? — dns spoofing." https://www.cloudflare.com/learning/dns/dns-cache-poisoning/, 2025. Accessed: 2026-2-22.

[7] Stephen J. Friedl, "An illustrated guide to dns cache poisoning." http://unixwiz.net/techtips/iguide-kaminsky-dns-vuln.html, 2008. Accessed: 2026-3-4.

[8] Imperva, "What is dns hijacking / redirection attack." https://www.imperva.com/learn/application-security/dns-hijacking-redirection/, 2026. Accessed: 2026-2-26.

[9] Ubuntu Server, "Installing dns security extensions." https://documentation.ubuntu.com/server/how-to/networking/install-dnssec/, 2026. Accessed: 2026-2-20.

[10] R. Arends, et. al, "Resource records for the dns security extensions." https://datatracker.ietf.org/doc/html/rfc4034, 2005. Accessed: 2026-2-23.

[11] BIND9, "Dnssec guide." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html, 2025. Accessed: 2026-2-25.

[12] dnsimple, "What are nsec and ncsec3 records?." https://support.dnsimple.com/articles/nsec-nsec3-records/, 2026. Accessed: 2026-4-23.

[13] BIND9, "Trust anchor." https://bind9.readthedocs.io/en/v9.18.39/dnssec-guide.html#trust-anchors, 2025. Accessed: 2026-2-25.

[14] dnsimple, "The dnssec chain of trust." https://support.dnsimple.com/articles/dnssec-chain-of-trust/, 2026. Accessed: 2026-2-28.

[15] Vicky Risk, Suzanne Goldlust, "Automatic dnssec zone signing key rollover explained." https://kb.isc.org/docs/aa-00822, 2021. Accessed: 2026-4-20.

[16] RFC 8624, "Algorithm implementation requirements and usage guidance for dnssec." https://www.rfc-editor.org/rfc/rfc8624#section-3.1, 2019. Accessed: 2026-4-23.

[17] Nawaz Dhandala, "How to choose the right dnssec algorithm (rsa vs ecdsa)." https://oneuptime.com/blog/post/2026-01-15-choose-dnssec-algorithm-rsa-ecdsa/view, 2026. Accessed: 2026-4-30.

[18] OpenSSL Project Authors, "Evp_digestsigninit." https://docs.openssl.org/master/man3/EVP_DigestSignInit/, 2024. Accessed: 2026-4-30.

[19] OpenSSL Project Authors, "Evp_digestverifyinit." https://docs.openssl.org/master/man3/EVP_DigestVerifyInit/, 2024. Accessed: 2026-4-30.

Appendix

A.1 Procedure Manual

Attached in the following pages is a procedure manual that outlines the early steps of getting the virtual environment set up, as well as configuring the DNS servers and enabling DNSSEC.

Steps:

1. Creating a VM
    a. First, I created a VM on VMWorkstation. I selected custom setup and kept most of the settings default. My method for adding the OS to the machine was to download the LTS version of Ubuntu to my desktop and using an ISO image, I copied it onto the machine. You can create an account for your machine and name your machine whatever you may like. I gave my vm 2gb of ram and 20 gb of disk (disk space is a few steps later), as this was the recommended minimum for Ubuntu.
    b. FOR THE TIME BEING, use a NAT network condition so you can download necessary things from the internet. The default options till the end is fine, using the recommended options is something I would recommend too.
    c. Resizing VM Disk space: I started with 20gb of disk, enough to comfortably install ubuntu and have some room to work. I wanted the vm to have 40 gb of disk space to have room for software and programs I would need to implement DNSSEC later. It is probably best to backup or take a snapshot of your vm before doing this step.
        i. Turn off the machine. Expand the disk in the VM settings, with expand disk to the desired size. Then, restart the machine and follow the commands below.
        ii. For commands, I ran:
            1. *lsblk* - I ran this to see where my root partition was, like /dev/sda1 or whichever says 20G. In the picture below, it is sda2, originally 20G, now 40G after resizing.

```
steve@steve-VMware-Virtual-Platform:~$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  73.9M  1 loop /snap/core22/2045
loop2    7:2    0 251.6M  1 loop /snap/firefox/7672
loop3    7:3    0  73.9M  1 loop /snap/core22/2216
loop4    7:4    0 251.7M  1 loop /snap/firefox/7720
loop5    7:5    0  11.1M  1 loop /snap/firmware-updater/167
loop6    7:6    0  18.5M  1 loop /snap/firmware-updater/210
loop7    7:7    0   516M  1 loop /snap/gnome-42-2204/202
loop8    7:8    0 531.4M  1 loop /snap/gnome-42-2204/247
loop9    7:9    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop10   7:10   0  49.3M  1 loop /snap/snapd/24792
loop11   7:11   0  10.8M  1 loop /snap/snap-store/1270
loop12   7:12   0   576K  1 loop /snap/snapd-desktop-integration/315
loop13   7:13   0  48.1M  1 loop /snap/snapd/25935
loop14   7:14   0   576K  1 loop /snap/snapd-desktop-integration/343
sda      8:0    0    40G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0    40G  0 part /
sr0     11:0    1   5.9G  0 rom  /media/steve/Ubuntu 24.04.3 LTS amd64
```

2. Next, ensure that you have cloud guest tools installed: *sudo apt-get update* and *sudo apt-get install cloud-guest-utils*
3. Next, expand the partition with *sudo growpart /dev/sda 1*, change the number to the partion number shown by lsblk.
4. Then run *sudo resize2fs /dev/sda<previous partion number here>* .
5. The run df -h to verify the new size

d. After reportioning, we can download some software before switching to a virtual network. For ubuntu, I downloaded BIND9 to set up my DNS Servers
   i. Command: *sudo apt install bind9* and *sudo apt install dnsutils*
      1. DNSutils provides helpful troubleshooting and debugging tools, and may not be installed with bind9

e. Also, install gcc to run c programs you might need later. If you would like to as well, install Wireshark on a VM as well to monitor and see the traffic you are sending.

f. VMTOOLS
   i. I tried numerous trouble shooting techniques to install VM Tools on the vm, but I ended up doing it through the command line instead of VMWare. I did not have administrative privileges on the machine and I think that is what prevented me from doing it through VMWare.
   ii. To install them, make sure you have an internet connection and run :
      1. *sudo apt-get install open-vm-tools-desktop*. Says yes when prompted, and once complete check the status with: *systemctl status vmtoolsd*. You should see “active” in green text.

g. At this point, cloning once for another machine to treat as a dns server is not the worst idea, I made the mistake of cloning a bit too early so I just had to repeat steps on multiple vms. It wouldn’t hurt either to set up the virtual network and make sure your virtual network can be accessed by the virtual machine.

2. Setting Up the Virtual Network
   a. Under the “Edit” Tab in the menu bar, there should be a Virtual Network Editor Option. Select this to begin setting up the network.
   b. Select the “Add Network Option to create your new network.
   c. Make the vmnet# one that is not taken yet. For me, 0, 1, and 8 were in use, so I used vmnet5. Select Host-only for the network type.

d. After selecting add and creating the network, you will be able to change specifics of the network. Enter the virtual machine settings and add a network adapter setting. Deselect DHCP service to distribute IP addresses to VMs. The IP addresses will be statically assigned by you. Connecting a host virtual adapter can also be deselected, it should not be needed to connect with the host on this network.
e. You can manually input a private IP address, like 192.168.x.0, or if you save, the editor will give you a random 192.168.x.0 IP address.

3. Cloning the VM
    a. Ensure the VM you intend to clone is powered off.
        i. Before cloning, ensuring that you have a Network Adapter that is connected to the vmnet you created in the previous step. To do this, navigate to the vm settings by right-clicking on the vm itself, clicking on the vm menu tab, or while it is turned off it can be seen while viewing the vm.
        ii. Under “Hardware”, select add at the bottom of the screen. Select Network Adapter and finish.
        iii. Next, select the newly added Network Adapter, likely called Network Adapter 2 if you still have the original NAT connection.
        iv. Select Custom and your vmnet# should be available to choose from.
        v. At this stage, you shouldn’t really need the NAT connection anymore, so you can remove it or deselect both of its connection options, “connected” and “connect at power on” so that it is easily available and can be turned on whenever, versus having to add it whenever you may need to access the internet. I would say it is best practice to delete it so that it cannot be defaulted to and you can ensure that your local network is working correctly.

4. Configuring the Machines IP
    a. On your Client Machine:
        i. Open a terminal and run the command:
            1. *ip a*. If you get an outpout like the one below, then you are likely also getting “Connection issues” popups. This is because the vm is recognizing your vmnet, but doesn’t have an address yet. To set one, open up the network settings of your OS and navigate to IPv4 tab. Select manual for the method and then input an address. It’ll match you vmnet, like 192.168.x.y, x being what you set or assigned your vmnet, and y being

whatever you like, between 1 and 255. I chose 130 to start, but its personal preference.

```
steve@steve-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:7e:de:ef brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.107.130/24 brd 192.168.107.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
```

2. Then, for the subnet mask, input the same number that is listed on the vmnet.

ii. Also on the client machine, you should edit the netplan configuration. See the below photo to see the format, and replace the addresses field with your own client IP, name server field with your own name server ip, and search with your own domains. Ens36 is my leftover from when I added a NAT connection to download vmtools, and isn't necessary but it isn't hurting anything.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.107.130/24
      nameservers:
        addresses:
          - 192.168.107.129
        search:
          - lab
          - jmu.lab

    ens36:
      dhcp4: yes
```

b. On the Primary Server:

i. Open a terminal and run the command:

1. *ip a*. If you get a similar output as before, then follow the same steps as above to set the IP and Subnet.

2. However, for the DNS Server address, put the local 127.0.0.1, showing that this machine is a DNS Sever. Below should be the output after you add this to the settings:

```
steve@steve-VMware-Virtual-Platform:/etc/bind$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub
       DNS Servers: 127.0.0.1

Link 2 (ens33)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 127.0.0.1
       DNS Servers: 127.0.0.1
```

3. The below netplan may or may not be necessary, use it as a last resort if you are still unable to connect at the end.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/netplan/01-network-manager-all.y
aml
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.107.129/24
    ens36:
      dhcp4: yes
```

c. Back on the Client Machine:

i. After setting the Primary Server IP, go back to the client and set the DNS in the same area of settings and put in the Primary Server address you just assigned.

ii. Try pinging the client vm from another machine. The primary server with a command like ping 192.168.107.130 to ping the client machine to check for connectivity.

iii. In a terminal, run the command *resolvectl status,* and you should see something like this below

```
steve@steve-VMware-Virtual-Platform:~$ resolvectl status
Global
         Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (ens33)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.107.129
       DNS Servers: 192.168.107.129
        DNS Domain: lab jmu.lab
```

d. On the Delegated Server
    i. Open a terminal and run:
        1. *Ip a*. For this machine, all you need to do is set the IP address and subnet mask as you did before. Try pinging both machines like you did before, and if the output is receiving packets from other machines, they are connected.

5. Setting up DNS on the VM
    a. Setting up a primary server.
        i. This server will be, as the name says, the primary server that will be used when querying DNS. I will use example.jmu.lab for setting up the DNS. A good name for this vm would be VM1.
        ii. First, you must add a zone. A zone in this instance would be “lab”. To add a zone, you must enter */etc/bind/named.conf.local* and edit it.
            1. The syntax is seen below, and a db file is assigned to this zone.

```
//
// Do any local configuration here
//

zone "lab" {
        type master;
        file "/etc/bind/db.lab";
};
```

        iii. The db file does not automatically appear, hower making a copy of db.local into the name you gave it in the zone, mine being “db.lab”. Now, nano into the newly created db file.
            1. With the db file, you see lines that start with an @. The first line contains a lot of info, the line with SOA (Start of Authority).

After the SOA, you need to edit the localhost part. For me, it would be lab. and root.lab.

2. Below that is the serial number, and this is an important part too. Every time you update this file, the serial number must be incremented too. It must only be incremented once if you make multiple changes before restarting bind9. The other numbers in the SOA aren't too important right now.
3. Next, you must edit the lines at the bottom. At a minimum, you need an example IN A 192.168.x.y, x being the network octet and y being an octet of your choosing.
4. To make the primary server an authoritative server, *@ IN NS ns.<domain>.lab.* line is needed.
5. A *ns.<domain>.lab. IN A 192.168.x.y* line, the x being your network ip and the y octet being whatever you like, but stay away from the ips you have reserved for the Client and DNS servers.
6. To make this server authoritative, the jmu.lab. and ns1.jmu.lab. lines are required, with the ns1.jmu.lab. line ending with the delegated server's IP address.
7. I haven't edited the top comment, and it might be misleading but that is the comment from db.local, please disregard.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.lab. root.lab. (
                              5         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.lab.

ns1     IN      A       192.168.107.129

jmu.lab.        IN      NS      ns1.jmu.lab.
ns1.jmu.lab.    IN      A       192.168.107.128
bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

8. When adding to the domain server, you will likely run into errors. I these are after changing named.conf file, run a command to check for those errors: named-checkconf. I you are having issues with a zone or named-checkconf return ok, the try named-checkzone <zone-name> /etc/bind/db.<zone-name>. If it tells you an error, correct it and and keep trying till you get an OK.

```
steve@steve-VMware-Virtual-Platform:/etc/bind$ named-checkzone jmu.lab db.jmu.la
b
zone jmu.lab/IN: loaded serial 6
OK
```

9. At this point, running a dig command at the primary server is a good idea. Using the domain you made above, run a dig command like so:
    a. *dig @<primary server ip> example.jmu.lab* (my example domain). As you can see below, the status area shows whether or not querying the server was successful. You should see NOERROR, but you might see SERVFAIL or NXDOMAIN

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 computer.lab

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 computer.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 50660
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 77f6f6477cf686730100000069a5d81f095a8d9a76c0eb43 (good)
;; QUESTION SECTION:
;computer.lab.                  IN      A

;; ANSWER SECTION:
computer.lab.           86400   IN      A       192.168.107.2

;; Query time: 1 msec
;; SERVER: 192.168.107.129#53(192.168.107.129) (UDP)
;; WHEN: Mon Mar 02 13:34:07 EST 2026
;; MSG SIZE  rcvd: 85
```

10. SERVFAIL likely means there is an error in your network configuration, but running the check discussed above in 4.c.iv on the server could prove useful.

11. NXDOMAIN means that the was no domain found (Non-existent Domain), maybe there was a typo, or once again run the check discussed above in 4.a.iii.8 on the server could prove useful.

b. Setting up the Secondary Server
   i. For setting up the secondary server, you follow the same steps.
   ii. First edit /etc/bind/named.conf.local to be like this for the delegated zone, .jmu in this case:

```
//
// Do any local configuration here
//

zone "jmu.lab" {
        type master;
        file "/etc/bind/db.jmu.lab";
};
```

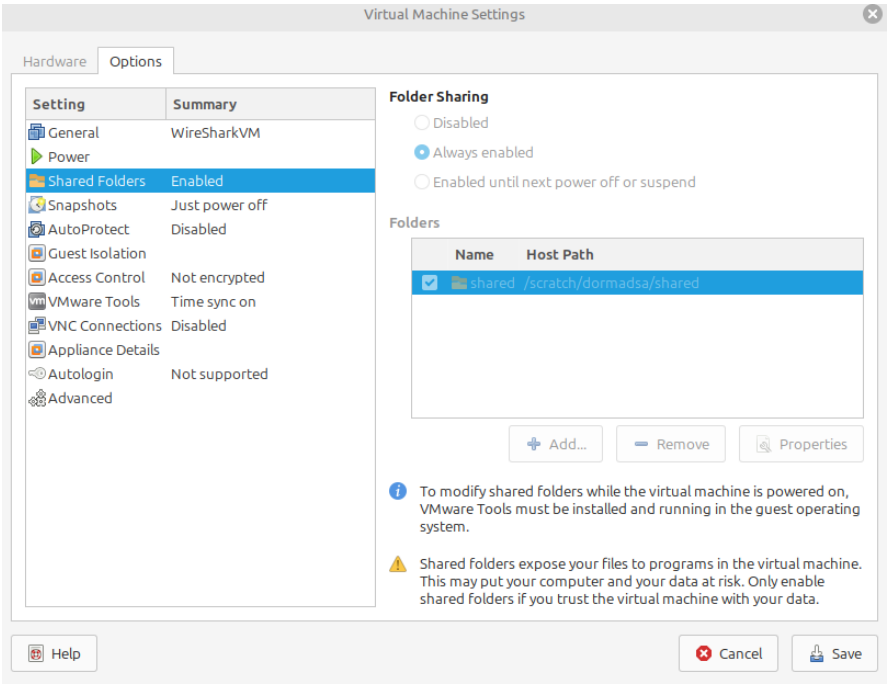
   iii. In this db file, the name space record points towards the server itself, and the other records are viable domains for <name>.jmu.lab.

```
steve@steve-VMware-Virtual-Platform:~$ cat /etc/bind/db.jmu.lab
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.jmu root.jmu (
                              6         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;

@       IN      NS      ns1.jmu.

ns1     IN      A       192.168.107.128
example IN      A       192.168.107.50
shea    IN      A       192.168.107.69
john    IN      A       192.168.107.67
alley   IN      A       192.168.107.68
king    IN      A       192.168.107.66
main    IN      A       192.168.107.65
quad    IN      A       192.168.107.64
```

6. Creating a shared folder between the VM and Host machine

a. https://community.broadcom.com/vmware-cloud-foundation/communities/community-home/digestviewer/viewthread?GroupId=7171&MessageKey=e83f0009-629f-42a2-b0d0-a9f6ea56b3d0&CommunityKey=fb707ac3-9412-4fad-b7af-018f5da56d9f

b. First, enter the setting of the VM you would like to remove information from. Configure the settings while the VM is off, the screenshot below was taken when the VM was on. As you can see below, switch over to the options tab, then go down to “Shared Folders”. Select Always enabled at the top, then “+ Add”. You can then set a name for the shared folder, as well as set the location you want it saved to on the host machine. I liked to keep it clean and make the name the same for both folders.



c. Then, start up the VM. Open a terminal and enter the command below. This will fuse the hgfs directory with the host, allowing you to then view your shared folder in the /mnt/hgfs directory. Now, you can copy or move files through the command line into this directory, and they will be put into the location you specified in the previous step on the host machine.

```
steve@steve-VMware-Virtual-Platform:~$ sudo vmhgfs-fuse .host:/ /mnt/hgfs -o all
ow_other
steve@steve-VMware-Virtual-Platform:~$ ls /mnt/hgfs
shared
```

7. Capturing Traffic with Wireshark
    a. I kept getting ntp.ubuntu network traffic, caused by my isolated network wanting to access the outside. To stop this, I ran *sudo timedatectl set-ntp off* to stop this bloat from coming through. This was sending hundreds of pings every couple of seconds, making it hard to filter out so I disabled it.

8. Configuring DNSSEC
    a. https://documentation.ubuntu.com/server/how-to/networking/install-dnssec/
    b. Keys are stored in /var/cache/bind when generated.
    c. Getting started – Parent
        i. Now that DNS is working, it is time to implement DNSSEC. As we will need to write to the disc, we have to move to another directory. I moved to /var/lib/bind/ to implement DNSSEC.
        ii. Firstly, copy over db.lab file from /etc/bind/ to /var/lib/bind/. You will need to edit it like so: We are going to add some delegation code here too, we are saving a step for later.

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
```

        iii. Still on the parent machine, nano into /etc/bind/named.conf.options.
            1. Outside of the options area, add a dnssec-policy like the one below. You need both a KSK and ZSK. The algorithm RSASHA256 can be swapped out for your algorithm of choice.

```
dnssec-policy rsa-policy {
        keys {
                ksk lifetime 365d algorithm RSASHA256 2048;
                zsk lifetime 60d algorithm RSASHA256 1024;
        };
};
```

iv. Then, edit /etc/bind/named.conf.local file field to point the "/var/lib/bind/db.lab". and add some additional lines, including the dnssec-policy <name> and inline-signing yes.

v. Restart bind on this machine.

vi. Run the command *dig @127.0.0.1 lab DNSKEY +dnssec +short* to just output the KSK for this zone. You will copy this key and paste it to the client machine. These will be carried into the next step.

d. On the client machine

i. Create a file to hold the trust anchor. I called mine lab-trust.key. I added it to "etc/bind/lab-trust.key". In the file, I added what is seen below. As seen below, the 257 recognizes this key as being the KSK for the lab zone. The base 64 key should be pasted at the end, with this format: "base-64 key";. The client must directly trust lab. as the top of the chain instead of a root server.

```
trust-anchors {
        lab. static-key 257 3 8 "AwEAAdyubupVYO97+QR5LxX3eIh8vxOI5bxfVz6ieZqMF19e
5Ul2SD+MyyY5iM1VRutYEpLmfbMDDuW3ZElJzN4BRP8huMuzRf8V4nAuB3poxIUs+kRywr4n6et46oCBF
7oWmcw792jAKKXFiuegwZ3s5NvrZufCckq5wzgaPvMxJLZLkalddzkc0KuvwAtmGsOTT7lkn3sx59JgWG
SYiL+3FNZFSZPlcDyw3QuIO1hN+Vq2NwuSR2LkOFX3miCKx4d07hr27TtP3LGJiBhcyMeJRT3i4bOBbCt
wGelA6xq5i4k1fg2+R2e+2VZtqk35GJsh8c0yqKoahz96ZCSrWCHm690=";
};
```

ii. Also, add the key file to your named.conf include path: <include_key>. This should be added near the other include paths in the file. See below:

```
include "/etc/bind/lab-trust.key";
```

iii. Then, to check the setup, *dig @parent_ip <domain> A +dnssec*. You should see and additional ad flag. If so,

e. On Child Server

   i. Firstly, copy over db.lab file from /etc/bind/ to /var/lib/bind/. You will need to edit it like so:

```
$TTL    86400   ; 1 day
jmu.lab.        IN      SOA     ns.jmu.lab. root.jmu.lab. (
                                10         ; Serial
                            43200         ; Refresh
                              900         ; Retry
                           1814400         ; Expire
                             7200 )       ; minimum
;

jmu.lab.        IN      NS      ns.jmu.lab.

ns.jmu.lab.     IN      A       192.168.107.128
example IN      A       192.168.107.50
shea    IN      A       192.168.107.69
john    IN      A       192.168.107.67
alley   IN      A       192.168.107.68
king    IN      A       192.168.107.66
main    IN      A       192.168.107.65
quad    IN      A       192.168.107.64
```

   ii. Then, in named.conf.local, edit /etc/bind/named.conf.local file field to point the "/var/lib/bind/db.lab". and add some additional lines, including the dnssec-policy <name> and inline-signing yes.

   iii. Restart bind.

f. Back on the client

   i. *dig @child_ip <domain> A +dnssec*. Yous should see an A record, RRSIG for A record (POSSIBLY KEYS?)

g. Back on Child Server

   i. Generate the Delegation Signer on Child and give it to the parent to link them.

      1. Run: *dnssec-dsfromkey -2 Kjmu.lab.+008+<KEYID>.key* that should produce something like: jmu.lab. IN DS 34939 8 2 <digest>

      2. Then, copy this data and save it for the next step on the parent server.

h. On the Parent

i. In /var/lib/bind/db.lab, add the following line for the DS record:

```
$TTL    86400
lab.    IN      SOA     lab. root.lab. (
                              3         ; Serial
                          43200         ; Refresh
                            900         ; Retry
                        1814400         ; Expire
                           7200 )       ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu     IN      NS      ns.jmu.lab.
ns.jmu  IN      A       192.168.107.128
; bc    IN      A       192.168.107.1
computer        IN      A       192.168.107.2
jmu IN DS 34939 8 2 9682051422A73107F882F98D04F47492ECA60648DC071428E43477AC7F13
42C2
```

i. EXTRA

i. To get DNSKEY and RRSIG files, I ran a *dig @server_address DNSKEY +dnssec*

ii. Running *sudo rndc flush* is not a bad idea to run if you run into errors with keys, especially if you end up regenerating keys.

Source: Ubuntu Server Documentation - https://documentation.ubuntu.com/server/how-to/networking/install-dns/ Date accessed: 1/20/2026

Bind9 Man Pages, Date accessed 1/20/2026

https://bind9.readthedocs.io/en/v9.20.17/chapter3.html

Resolvctl Man Pages, Date accessed: 1/26/2026

https://www.freedesktop.org/software/systemd/man/latest/resolvectl.html

Resolved Man Pages, Date accessed: 1/27/2026

https://www.freedesktop.org/software/systemd/man/latest/systemd-resolved.service.html